RECRUITMENT
APPLICANT TRACKING SYSTEM (ATS)

PROJECT DOCUMENTATION

Frontend: HTML, CSS, JavaScript and React

Backend: Python and FastAPI

Database: SQL / PostgreSQL

Prepared by: _____

Department: Computer Science and Engineering

Institution: _____

Academic Year: 2026–2027

Expanded academic project report • Version 2.0 • October 2026

1. Project Overview

The Recruitment Applicant Tracking System (ATS) is a web application that helps an organisation manage job openings and candidate applications from one place. It replaces scattered email threads and spreadsheets with a structured process. Recruiters can publish vacancies, review candidate profiles, move applications through hiring stages, arrange interviews, record feedback and prepare offers. Candidates can browse published jobs and submit applications through a careers page.

Example / key points

- Project title: Recruitment Applicant Tracking System (ATS)
- Frontend: HTML, CSS, JavaScript and React
- Backend: Python 3.12 and FastAPI
- Database: PostgreSQL using SQL
- Optional supporting services: Redis, Celery, email service and Docker

2. Abstract

Recruitment is an important business activity, but manual hiring processes can be slow and difficult to track. This project proposes a centralised ATS that organises job information, candidate details, application stages, interview schedules and hiring reports. The system provides separate access for administrators, recruiters, hiring managers, interviewers and candidates. It supports searchable records, a consistent workflow, validation and audit history. The expected result is better visibility, reduced duplicate data entry and a more organised candidate experience.

Example / key points

- The project is designed as a beginner-friendly academic implementation.
- Core features can be built first, then extended with authentication, reports and notifications.
- The report describes design choices and example code; production use requires additional testing and configuration.

3. Introduction to Recruitment

Recruitment includes identifying a vacancy, publishing a job description, collecting applications, screening candidates, arranging interviews and selecting a suitable applicant. Each step creates information that must be stored accurately. If records are maintained manually, a recruiter may lose track of a resume, miss an interview date or forget to inform a candidate about a decision. A digital ATS provides one consistent location for the recruitment lifecycle.

Example / key points

- Job requisition and job posting
- Candidate registration and resume submission
- Application review and stage tracking
- Interview planning and evaluation
- Offer decision and recruitment analytics

4. Problem Statement

Small and medium-sized organisations may rely on spreadsheets, email folders and shared documents to coordinate hiring. These tools are not always connected, making it difficult to know which candidate belongs to which vacancy or who last changed an application status. Duplicate candidate records and delayed communication can reduce efficiency. The proposed ATS addresses these problems through relational data storage, controlled workflow transitions, role-based access and searchable dashboards.

Example / key points

- Scattered records make information difficult to locate.
- Duplicate applications can create confusion.
- Interview feedback may be stored in inconsistent formats.
- Managers may lack a reliable view of pipeline progress.
- Reports may require manual counting and repeated spreadsheet updates.

5. Project Objectives

The principal objective is to create a maintainable recruitment portal that supports the major steps of hiring. The design separates the user interface, application rules and database so that each layer can be developed and tested independently. The system should make the hiring pipeline understandable to both technical and non-technical users.

Example / key points

- Centralise jobs, candidates, applications and interviews.
- Provide a defined application lifecycle: Applied, Screening, Interview, Offer, Hired or Rejected.
- Allow authorised staff to add notes and interview feedback.
- Prevent duplicate applications to the same job.
- Provide reports for pipeline totals and time-to-hire.
- Protect candidate data with authentication and permissions.

6. Project Scope

The initial scope includes job posting management, a public careers page, candidate application forms, resume upload, recruiter dashboards, interview scheduling, feedback, offers and basic reports. The system is intended to demonstrate a complete workflow using common web technologies. Features such as AI ranking, video interviews, payroll integration and native mobile applications are treated as possible future extensions rather than mandatory beginner requirements.

Example / key points

- Included: job CRUD, application submission, status tracking and basic reporting.
- Included: role-aware staff access and candidate contact details.
- Optional extension: email notifications and background task queue.
- Future scope: onboarding, calendar integration and advanced analytics.

7. Stakeholders and Roles

Different users need different capabilities. An administrator manages users and system settings. A recruiter creates job posts and coordinates applications. A hiring manager reviews candidates and approves offers. An interviewer sees assigned interviews and submits feedback. A candidate views public vacancies and submits an application. Keeping these roles separate reduces accidental changes and protects private recruitment information.

Example / key points

- Admin: manage users, roles, departments and configuration.
- Recruiter: create vacancies, review applications and schedule interviews.
- Hiring manager: review assigned candidates and approve offers.
- Interviewer: access assigned interviews and submit scorecards.
- Candidate: browse vacancies and apply.

8. Functional Requirements

Functional requirements describe what the application must do. Each requirement should be testable and traceable to a module. For example, a candidate application requirement can be verified by submitting valid details and confirming that an application record appears in the database. A status transition requirement can be verified by checking both the new stage and the stage-history record.

Example / key points

- FR-01: authorised users can sign in.
- FR-02: recruiters can create, edit, publish and close jobs.
- FR-03: candidates can submit a form and resume.
- FR-04: staff can search and filter applications.
- FR-05: interviewers can submit structured feedback.
- FR-06: the system can create and track offers.
- FR-07: reports show pipeline counts and hiring duration.

9. Non-Functional Requirements

Non-functional requirements describe quality attributes rather than individual features. The interface should be responsive and understandable. API endpoints should return useful validation errors. Database operations should use parameterised queries or an ORM. Access control should be enforced on the server, not only by hiding buttons in the browser. Backups and monitoring should be planned before the system is used with real candidate data.

Example / key points

- Security: password hashing, HTTPS and least-privilege permissions.
- Usability: labelled forms, readable status indicators and clear messages.
- Performance: pagination and indexes for larger lists.
- Reliability: database backups and error logging.
- Maintainability: separate routers, services, schemas and models.

10. Use Case: Apply for a Job

The candidate opens the careers page and chooses a published vacancy. The application form asks for a name, email, phone number, cover letter and resume. The browser performs basic checks, while the backend repeats all important validation. The server checks that the job is still open, validates the file type and size, detects duplicate applications and saves the candidate and application records. The user receives a confirmation message when the request succeeds.

Example / key points

- Precondition: the job exists and is published.
- Main flow: select job → complete form → upload resume → submit.
- Success: application is saved with stage Applied.
- Alternative: invalid file, missing field or duplicate application returns an error.

11. Use Case: Move an Application

A recruiter opens the pipeline board and selects an application. The user chooses a permitted next stage, such as Applied to Screening or Screening to Interview. The backend checks the current stage and role, writes a history entry and commits the change. If a move is invalid, the server returns a clear error and leaves the previous stage unchanged. This prevents the interface from becoming the only source of workflow rules.

Example / key points

- Precondition: recruiter is authenticated and may access the job.
- Validate the proposed transition on the server.
- Record old stage, new stage, user and timestamp.
- Send a notification only after a successful transaction.

12. System Architecture

The ATS uses a three-tier architecture. The presentation tier is a React single-page application running in the browser. The application tier is a Python FastAPI service that validates requests and applies recruitment rules. The data tier is PostgreSQL, which stores users, jobs, candidates, applications, interviews, feedback and offers. Resume files may be stored in a dedicated upload directory or object storage. Optional Redis and Celery services handle background work such as reminders.

Example / key points

- Browser / React UI → HTTPS API request → FastAPI route.
- FastAPI route → service layer → SQLAlchemy/database.
- Database response → JSON response → React component.
- Resume files should not be exposed through an unrestricted public folder.

13. Technology Stack

HTML defines the structure of web pages, CSS controls presentation and JavaScript adds interaction. React divides the interface into reusable components. Python and FastAPI provide the backend API and request validation. PostgreSQL stores structured data and enforces relationships. SQLAlchemy maps Python classes to database tables, while Alembic tracks schema changes. Pytest can test backend logic and React Testing Library can test user interface behaviour.

Example / key points

- Frontend: HTML5, CSS3, JavaScript ES2022, React and Vite.
- Backend: Python 3.12, FastAPI, Pydantic and SQLAlchemy.
- Database: PostgreSQL with relational constraints and indexes.
- Testing: Pytest, TestClient and React Testing Library.
- Development tools: Git, GitHub, VS Code or PyCharm.

14. Development Methodology

An iterative approach is suitable for a student project. Start with a small working feature and expand it after testing. First build the public jobs page, then create the application form, backend endpoints and database tables. Next add recruiter tools, interviews and reports. Each iteration should include a short review of the user flow, a code check and a test run. This approach makes errors easier to isolate than building every module at once.

Example / key points

- Iteration 1: requirements, wireframes and schema.
- Iteration 2: public careers page and job listing API.
- Iteration 3: candidate application submission.
- Iteration 4: recruiter pipeline and role controls.
- Iteration 5: interviews, offers, reports and testing.

15. HTML Frontend Fundamentals

HTML provides semantic structure for the careers page and application form. Use headings in a logical order, labels connected to inputs, and buttons with clear names. Semantic elements such as header, nav, main, section and footer improve readability and accessibility. HTML validation helps users before the form is submitted, but it does not replace backend validation because requests can be sent without using the browser interface.

Example / key points

- Use <form> for application submission.
- Use type=email for email fields and required for mandatory fields.
- Use an explicit label for every input.
- Use accept=".pdf,.docx" as a user-interface hint, not a security boundary.

16. CSS and Responsive Layout

CSS controls spacing, colours, typography and layout. A recruitment portal should be usable on desktop, tablet and mobile screens. Flexbox works well for navigation and simple rows, while CSS Grid can organise dashboard cards. Use consistent spacing and visible focus states. Avoid communicating application stages through colour alone; include text labels so the meaning remains clear for users with colour-vision differences.

Example / key points

- Use a centred content container and a readable maximum width.
- Use responsive grid columns for dashboard cards.
- Provide visible hover and keyboard focus styles.
- Check the application form at narrow screen widths.

17. JavaScript Fundamentals

JavaScript handles browser events and can validate form values before sending requests. Keep the code modular: one function can validate inputs, another can submit the form and another can display a status message. Avoid inserting untrusted values using innerHTML. When displaying candidate-supplied text, use React's normal text rendering or safe DOM text properties. Treat all browser input as untrusted until validated by the backend.

Example / key points

- Use const by default and let when reassignment is needed.
- Use addEventListener for browser event handling.
- Use fetch or an API client to communicate with the backend.
- Display success and error messages in a clear status region.

18. React Component Design

React components represent reusable interface elements such as a navigation bar, job card, application form, stage badge and dashboard metric card. Components receive props and maintain local state where needed. Keep API calls in a service or API-client module rather than duplicating request logic throughout the interface. For larger applications, React Router manages page navigation and a query library can cache server data.

Example / key points

- App.jsx defines route-level structure.
- JobCard.jsx displays a single vacancy.
- ApplyForm.jsx manages candidate form state.
- PipelineBoard.jsx groups applications by stage.
- Dashboard.jsx displays recruitment metrics.

19. Frontend Validation

Validation improves usability by catching simple mistakes before submission. The form should check that required values are present, email syntax is plausible and the resume is within the size limit. Validation messages should explain how to correct the issue. Server validation remains authoritative: the backend must check the file extension, content type, maximum size, job status and duplicate application rule even if the browser reports that everything is valid.

Example / key points

- Check required name and email fields.
- Limit resume uploads to the allowed formats and size.
- Disable repeated submission while a request is pending.
- Preserve entered values after recoverable errors.

20. Frontend API Integration

The frontend communicates with FastAPI through HTTP requests. A central API client can define the base URL and shared headers. Requests should handle loading, success and failure states. In production, use environment variables for the API base URL and avoid committing secrets to the repository. Authentication tokens should be stored and handled according to the application's security design; for production systems, secure HttpOnly cookies are often preferable to localStorage.

Example / key points

- GET retrieves jobs or applications.
- POST creates jobs or submits applications.
- PATCH updates an application stage.
- PUT replaces a resource where that API design is appropriate.
- Show a useful message when the network or server is unavailable.

21. Python Backend Fundamentals

Python is used for backend logic because its syntax is readable and its ecosystem supports web APIs, data access and testing. Organise the backend into modules instead of placing all logic in one file. Routes handle HTTP input and output, schemas validate data, services apply business rules and repositories or ORM sessions interact with the database. This separation makes the code easier to understand and test.

Example / key points

- Use virtual environments to isolate dependencies.
- Define functions with clear names and return values.
- Use exceptions for invalid operations and map them to HTTP errors.
- Keep configuration outside source code where possible.

22. FastAPI Request Handling

FastAPI maps URL paths and HTTP methods to Python functions. Pydantic models describe the expected JSON body and validate fields. Dependency injection can provide a database session or current user. The interactive API documentation helps developers try endpoints during development. Endpoints should use suitable HTTP status codes, avoid returning password hashes and limit the amount of data returned in list operations.

Example / key points

- POST /api/v1/jobs creates a vacancy.
- GET /api/v1/public/jobs lists published vacancies.
- POST /api/v1/public/jobs/{id}/apply submits an application.
- PATCH /api/v1/applications/{id}/stage moves an application.

23. SQL and Relational Databases

PostgreSQL stores related entities in tables. A primary key identifies each row and a foreign key connects related records. For example, each application refers to one job and one candidate. Unique constraints prevent a candidate from applying to the same job more than once. Check constraints restrict stage values to an approved set. Indexes improve common searches, but should be selected based on actual query patterns.

Example / key points

- users and roles control staff access.
- jobs stores vacancy details.
- candidates stores candidate profile information.
- applications links jobs and candidates.
- interviews, feedback and offers extend the hiring workflow.

24. Database Normalisation

Normalisation reduces unnecessary duplication and update anomalies. The ATS separates candidates from applications because one candidate may apply to different jobs. It separates interviews from feedback because an interview can collect multiple interviewer scorecards. It separates stage history from the current application stage so that prior changes remain available for audit and reporting. The schema should balance integrity, clarity and query performance.

Example / key points

- Store each candidate's core contact details once.
- Use foreign keys instead of repeating job descriptions in applications.
- Use a history table for stage changes.
- Add indexes for frequently filtered columns after reviewing query plans.

25. Authentication and Authorisation

Authentication determines who a user is. Authorisation determines which actions that user may perform. A login endpoint verifies a password against a stored password hash and returns a session or token. Every protected backend endpoint must verify identity and permissions. Hiding a menu item in React is not sufficient because a user can call the API directly. Role checks should be combined with record ownership checks where needed.

Example / key points

- Hash passwords with a suitable password-hashing library.
- Use HTTPS in deployed environments.
- Give users only the permissions required for their role.
- Rate-limit login attempts and log suspicious activity.
- Never store plaintext passwords.

26. Resume Upload Handling

Resume uploads require special care because files come from external users. The backend should enforce a maximum size, check extension and MIME type, generate a random stored filename and keep files outside the public web root. File scanning can be added for production. Store the original filename only if needed for display and escape it when rendering. Access to a stored resume should require authorisation.

Example / key points

- Allow only approved file types such as PDF and DOCX.
- Reject oversized or empty files.
- Do not trust the browser's accept attribute.
- Avoid using user-provided filenames as filesystem paths.
- Apply access controls to resume download endpoints.

27. Application Pipeline

The pipeline provides a clear view of progress. Typical stages are Applied, Screening, Interview, Offer, Hired and Rejected. The backend should define allowed transitions and validate each change. A rejected application may be reopened only if that behaviour is explicitly allowed by the organisation. Each transition should create a history record with the previous stage, new stage, user and timestamp.

Example / key points

- Applied → Screening
- Screening → Interview
- Interview → Offer
- Offer → Hired
- Any permitted stage → Rejected with a reason
- Rejected → Screening only if reopening is supported.

28. Interview Scheduling

Interview scheduling stores the round name, date and time, duration, mode and panel members. The user interface should show time zones clearly where interviews involve different locations. The backend should validate that required participants exist and that the application is still active. Reminder notifications may be sent through a background worker. Changes and cancellations should be visible to authorised staff.

Example / key points

- Round examples: phone screen, technical, HR and final.
- Modes: onsite, video or phone.
- Store interview status: scheduled, completed, cancelled or no-show.
- Provide feedback forms to assigned interviewers only.

29. Interview Feedback

A structured scorecard helps interviewers record consistent observations. Criteria might include technical knowledge, communication and role-related skills. A numeric rating should always be accompanied by comments where useful. Feedback is sensitive recruitment information and should not be exposed to candidates or unrelated staff. The system should record who submitted feedback and when, and prevent accidental duplicate submissions for the same interview and interviewer.

Example / key points

- Use a defined rating scale, such as 1 to 5.
- Provide recommendation options with clear meanings.
- Allow comments that relate to job requirements.
- Restrict access by role and interview assignment.

30. Offer Management

An offer record contains the proposed salary, joining date, expiry date and approval status. A recruiter may draft an offer, while a hiring manager approves it before it is sent. The system should track accepted, declined or expired outcomes. Salary and joining-date data should be visible only to authorised users. Business rules should prevent an offer from being issued for a rejected application.

Example / key points

- Validate salary and date fields.
- Track pending, approved, sent, accepted, declined and expired states.
- Record the approver and approval timestamp.
- Keep a history of significant offer changes.

31. Search and Filtering

Search and filtering help recruiters find records quickly. Common filters include job title, candidate name, stage, source, application date and interview rating. Use server-side pagination for large result sets so that the browser does not need to load every record. Search input should be parameterised and length-limited. A useful interface shows active filters and allows users to clear them.

Example / key points

- Filter jobs by publication status and department.
- Filter applications by stage and vacancy.
- Search candidates by name or skill.
- Use pagination and stable sorting.
- Create indexes only for measured query needs.

32. Dashboard and Reports

The dashboard summarises recruitment activity. Useful metrics include open jobs, applications per stage, scheduled interviews, pending offers, source effectiveness and average time-to-hire. Reports must use clear definitions. For example, time-to-hire can mean the number of days from application submission to the date the application enters Hired. Reports should state their date range and handle missing or incomplete history carefully.

Example / key points

- Pipeline totals support workload planning.
- Source reports help compare careers pages, referrals and job boards.
- Time-to-hire supports process improvement.
- Offer acceptance rate helps assess offer effectiveness.
- Do not interpret small samples as definitive trends.

33. Notifications

Notifications keep candidates and staff informed about important events. Typical events include application received, interview scheduled, stage changed, offer issued and rejection. Email content should be concise and accurate. Avoid including unnecessary sensitive data. Background workers can send email without making the main request wait for the mail server. The system should log delivery failures and retry transient errors carefully.

Example / key points

- Use approved templates with placeholders.
- Store SMTP settings in environment variables.
- Do not send duplicate notifications after retries.
- Make email delivery status observable to administrators.

34. Audit Logging

Audit logs record significant actions for troubleshooting and accountability. Each entry should identify the actor, entity, entity identifier, action, timestamp and relevant change summary. Do not log passwords, authentication tokens or full resume contents. Audit logs should have restricted access and retention rules. For high-integrity use cases, consider append-only storage and monitoring for unusual activity.

Example / key points

- Log job publication and closure.
- Log application stage transitions.
- Log offer approval and status changes.
- Include request identifiers where practical.
- Protect logs from unauthorised modification.

35. Testing Strategy

Testing should cover business rules, API contracts, frontend interactions and end-to-end workflows. Unit tests check small functions in isolation. Integration tests verify that API endpoints work with a test database. Frontend tests confirm validation and user messages. End-to-end tests simulate a candidate applying and a recruiter moving the application through the pipeline. Security and load testing can be introduced as the system matures.

Example / key points

- Test valid and invalid form submissions.
- Test duplicate applications.
- Test invalid stage transitions.
- Test role restrictions and file-size limits.
- Test report calculations using known sample data.

36. Deployment and Maintenance

For local development, the frontend and backend may run as separate processes. A production deployment should use HTTPS, managed secrets, a production database, backups and centralised logs. Docker can make development environments more repeatable. Database migrations should be reviewed before they are applied. A deployment checklist should include environment variables, CORS origins, storage permissions, email configuration and rollback planning.

Example / key points

- Build frontend assets for production.
- Run FastAPI behind a suitable ASGI server.
- Apply database migrations in a controlled release.
- Back up the database and test restoration.
- Monitor API errors, latency and failed background jobs.

37. Project Limitations

A beginner implementation will not cover every complexity of enterprise recruitment. Resume parsing may be manual, integrations may be limited and notifications may require SMTP configuration. Reports can be inaccurate if stage history is incomplete. Security controls must be reviewed before using real personal information. These limitations should be documented honestly in an academic report and used to identify future improvements.

Example / key points

- No automatic guarantee of candidate-job fit.
- Email delivery depends on external mail configuration.
- Resume content is not proof of candidate qualifications.
- The initial system may not integrate with payroll or onboarding.
- Production readiness requires privacy, security and performance review.

38. Future Enhancements

Future development can improve the ATS after core workflows are stable. Calendar integration can reduce scheduling conflicts. A candidate portal can show application progress. Analytics can compare hiring sources and identify bottlenecks. Resume parsing may extract skills for recruiter review, but any automated recommendation should be explainable and reviewed by humans. Accessibility, multilingual support and mobile usability can also be improved.

Example / key points

- Calendar and video meeting integration.
- Candidate self-service status page.
- Advanced analytics and configurable reports.
- Resume parsing with human review.
- Integration with onboarding and HR systems.
- Accessibility audits and multilingual interface.

39. Conclusion

The Recruitment Applicant Tracking System demonstrates how a modern web application can organise recruitment from vacancy publication to hiring decision. React and JavaScript support the user interface, FastAPI and Python enforce application rules, and PostgreSQL maintains structured records. Role-based access, validation, audit history and tests are important parts of the design. The project can be implemented incrementally, beginning with job listings and applications and extending to interviews, offers and reports.

Example / key points

- The project reduces dependence on disconnected spreadsheets.
- A relational model keeps job and candidate records consistent.
- The backend remains responsible for security and workflow rules.
- Testing and documentation support maintainability.
- Future work can extend the system without changing its core purpose.

40. Project Title and Cover Page

Use the official title Recruitment Applicant Tracking System (ATS). Add the student name, degree, department, college, academic year, guide name and submission date. Keep the cover page formal and uncluttered. Do not add personal details that are not required by the college format.

Example / key points

- Project title
- Student name and roll number
- Department and institution
- Guide / supervisor
- Academic year

41. Certificate Page

A certificate page is commonly included in academic project reports. Replace the placeholders with details approved by the department. Do not claim that the project has been certified or approved until the relevant faculty member has signed it.

Example / key points

- Project title and student name
- Department and institution
- Guide signature
- Head of department signature
- Date and place

42. Declaration

The declaration states that the report is the student's own academic work and acknowledges any references, libraries or templates used. Follow the institution's required wording. Avoid claiming originality for third-party code or libraries; cite them appropriately.

Example / key points

- Use your institution's approved declaration format.
- List external libraries and references.
- Do not include signatures until authorised.

43. Acknowledgement

The acknowledgement thanks the project guide, faculty, institution and anyone who provided legitimate support. Keep the language professional and concise. Personal acknowledgements should be included only with the student's consent.

Example / key points

- Thank the project guide.
- Acknowledge faculty and department support.
- Mention peer assistance if relevant.
- Avoid unsupported claims about sponsorship.

44. Table of Contents

The table of contents should mirror the report headings and page numbers. In a word processor, generate it automatically from heading styles so that it can be refreshed after edits. A long report should also include lists of figures, tables and abbreviations.

Example / key points

- Use consistent heading levels.
- Refresh page numbers before submission.
- Keep figure and table captions consistent.

45. List of Figures

Figures can include the system architecture, use-case diagram, entity relationship diagram, application pipeline, login page, careers page, application form, recruiter dashboard and test results. Every figure should have a number, caption and short explanation in the body of the report.

Example / key points

- Figure 1: Three-tier architecture
- Figure 2: ATS use-case overview
- Figure 3: Database relationship diagram
- Figure 4: Recruitment pipeline
- Figure 5: Dashboard screen

46. List of Tables

Tables are useful for requirements, user roles, database fields, API endpoints, test cases and project schedules. Use descriptive captions and avoid screenshots of tables when selectable text is possible. Ensure each table is mentioned in the report.

Example / key points

- Requirements traceability
- Role permissions
- Database schema
- API endpoint summary
- Test cases and outcomes

47. Glossary

A glossary defines technical terms used in the report so that readers can understand the design without repeatedly searching for definitions. The terms should be consistent throughout the document.

Example / key points

- ATS: Applicant Tracking System
- API: Application Programming Interface
- ORM: Object Relational Mapper
- RBAC: Role-Based Access Control
- JWT: JSON Web Token
- CRUD: Create, Read, Update, Delete

48. Abbreviations

Abbreviations should be expanded at first use and used consistently afterward. Avoid introducing an abbreviation that appears only once unless it is a widely recognised technical term.

Example / key points

- UI – User Interface
- UX – User Experience
- SQL – Structured Query Language
- REST – Representational State Transfer
- HTTPS – Hypertext Transfer Protocol Secure
- CI/CD – Continuous Integration / Continuous Delivery

49. Feasibility Study

A feasibility study evaluates whether the project can be developed with available time, skills and tools. The stack uses widely available development tools and open-source libraries. The main cost considerations are hosting, email delivery and secure file storage if the application is deployed publicly.

Example / key points

- Technical feasibility: supported frameworks and relational database.
- Operational feasibility: familiar browser-based interface.
- Economic feasibility: open-source development stack.
- Schedule feasibility: implement in incremental milestones.

50. Risk Analysis

Risk analysis identifies problems that could affect delivery or safe operation. Each risk should have a likelihood, impact and mitigation. The highest priorities include unauthorised access to candidate data, unsafe file uploads, accidental data loss and incomplete workflow validation.

Example / key points

- Risk: unauthorised access — enforce server-side role checks.
- Risk: data loss — maintain and test backups.
- Risk: invalid files — validate type and size on the server.
- Risk: missed notifications — log failures and retry carefully.

51. Requirements Traceability

Traceability links each requirement to a module and test case. This helps show that the implementation addresses the stated problem. For example, the duplicate-application requirement maps to a unique database constraint and an API test that attempts to apply twice.

Example / key points

- FR-02 job management → Jobs API → job CRUD tests.
- FR-03 applications → Apply API → form and duplicate tests.
- FR-05 interviews → Interview API → scheduling tests.
- FR-09 reports → Reports API → metric calculation tests.

52. User Interface Wireframes

Before writing components, sketch the page structure. A careers page can show the logo, search field, job cards and Apply button. The recruiter dashboard can show metric cards, recent applications and a pipeline chart. Wireframes should focus on information hierarchy rather than final colours.

Example / key points

- Careers page: search, filters and job cards.
- Application form: personal details, resume and cover letter.
- Recruiter dashboard: metrics and pipeline.
- Candidate detail: profile, timeline, notes and interviews.

53. Use-Case Diagram Guidance

A use-case diagram represents actors and the actions they perform. Draw Candidate outside the system boundary with Browse Jobs and Apply for Job use cases. Draw Recruiter with Manage Jobs, Review Applications and Schedule Interview. Hiring Manager can Review Candidates and Approve Offers. Interviewer can View Assigned Interview and Submit Feedback.

Example / key points

- Actors: Admin, Recruiter, Hiring Manager, Interviewer, Candidate.
- Use clear action names beginning with verbs.
- Keep the system boundary labelled ATS.
- Do not use the diagram as a replacement for written use cases.

54. ER Diagram Guidance

The entity relationship diagram shows how the database tables relate. A Job has many Applications. A Candidate can have many Applications. Each Application can have multiple Interviews and Notes. An Interview can collect multiple Feedback records. An Application may have one Offer. Use crow's-foot notation or another consistent notation and show key relationships.

Example / key points

- Job 1—many Applications
- Candidate 1—many Applications
- Application 1—many Interviews
- Interview 1—many Feedback entries
- Application 1—zero-or-one Offer

55. Data Dictionary: Jobs

The jobs table stores vacancy information. Use a primary key for the job identifier and a foreign key for the department. The employment type and status fields should use controlled values. A closing date can be nullable for vacancies that do not have a fixed deadline. Store salary bounds consistently and validate that the minimum is not greater than the maximum.

Example / key points

- id: primary key
- title: required job title
- department_id: optional foreign key
- employment_type: full-time, part-time, contract or intern
- status: draft, published, closed or archived
- closing_date: optional deadline

56. Data Dictionary: Candidates

The candidates table stores the person's core profile. Email is useful for duplicate detection and should be normalised consistently, such as by trimming whitespace and converting to lowercase. Skills can be stored as a separate relation or, for a simple PostgreSQL design, an array. Choose one representation deliberately and document the trade-off.

Example / key points

- id: primary key
- full_name: required display name
- email: unique contact identifier
- phone: optional contact number
- experience_years: optional numeric value
- resume_path: protected file reference

57. Data Dictionary: Applications

The applications table connects a candidate to a job. It stores the current stage, cover letter, application timestamp and optional rejection reason. A unique constraint on job_id and candidate_id prevents repeated applications to the same vacancy. The application stage history is stored separately so that previous decisions remain available.

Example / key points

- id: primary key
- job_id: foreign key to jobs
- candidate_id: foreign key to candidates
- stage: controlled pipeline value
- applied_at: timestamp
- UNIQUE(job_id, candidate_id)

58. Data Dictionary: Interviews

The interviews table stores a scheduled round associated with an application. It should contain the date/time, duration, interview mode, meeting link if applicable and status. Panel membership is a many-to-many relationship and can be stored in a join table. Ensure that interviewers can only access interviews assigned to them.

Example / key points

- application_id: foreign key
- round_name: phone, technical, HR or final
- scheduled_at: timestamp with time zone
- duration_min: positive duration
- mode: onsite, video or phone
- status: scheduled, completed, cancelled or no-show

59. Data Dictionary: Feedback

Feedback records provide a consistent structure for interview evaluation. Score fields should be constrained to a valid range. The recommendation should be chosen from a defined set. A uniqueness constraint on interview_id and interviewer_id can prevent duplicate scorecards by the same interviewer for one interview.

Example / key points

- technical_score: 1–5
- communication_score: 1–5
- culture_score: 1–5
- recommendation: strong yes, yes, no or strong no
- comments: optional text
- submitted_at: timestamp

60. SQL: Create Roles Table

A roles lookup table prevents each user record from storing arbitrary role names. The role identifier is the primary key and the role name is unique. Insert the roles required by the application before creating staff users.

Example / key points

```
CREATE TABLE roles (
  id SMALLSERIAL PRIMARY KEY,
  name VARCHAR(30) UNIQUE NOT NULL
);
```

61. SQL: Create Users Table

The users table stores staff accounts. Passwords must be stored as password hashes, never as plain text. The role_id foreign key links each user to a permitted role. Active status supports account deactivation without deleting historical actions.

Example / key points

```
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role_id SMALLINT NOT NULL REFERENCES roles(id),
  is_active BOOLEAN NOT NULL DEFAULT TRUE
);
```

62. SQL: Create Jobs Table

A job table represents each vacancy. Use a controlled employment type and publication status. The description can be stored as text, while frequently filtered values should have suitable indexes. Validate salary bounds and closing date in application logic as well as through database constraints where practical.

Example / key points

```
CREATE TABLE jobs (
  id BIGSERIAL PRIMARY KEY,
  title VARCHAR(150) NOT NULL,
  employment_type VARCHAR(20) NOT NULL,
  description TEXT NOT NULL,
  status VARCHAR(15) NOT NULL DEFAULT 'draft',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

63. SQL: Create Candidates Table

Candidate profiles should be separate from applications because one person may apply for more than one job. Normalise email before inserting it and apply a unique constraint to reduce duplicate records. Restrict access to contact information and resume paths.

Example / key points

```
CREATE TABLE candidates (
  id BIGSERIAL PRIMARY KEY,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  phone VARCHAR(25),
  resume_path VARCHAR(300),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

64. SQL: Create Applications Table

The application record joins one job and one candidate. The stage column should use a controlled list of values. The unique constraint prevents the same candidate from applying twice to the same job. A separate history table captures every transition.

Example / key points

```
CREATE TABLE applications (
  id BIGSERIAL PRIMARY KEY,
  job_id BIGINT NOT NULL REFERENCES jobs(id),
  candidate_id BIGINT NOT NULL REFERENCES candidates(id),
  stage VARCHAR(20) NOT NULL DEFAULT 'applied',
  applied_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE(job_id, candidate_id)
);
```

65. SQL: Count Applications by Stage

Grouping by stage provides a simple recruitment pipeline report. Filter by job when the dashboard represents one vacancy. The query should use a bound parameter in application code rather than concatenating user input into SQL.

Example / key points

```
SELECT stage, COUNT(*) AS total
FROM applications
WHERE job_id = :job_id
GROUP BY stage
ORDER BY stage;
```

66. SQL: Find Candidates by Skill

Skill search can support recruiter discovery. The exact query depends on how skills are stored. If a PostgreSQL text array is used, the containment operator can check that the array contains requested values. If skills are stored in a separate table, use joins instead.

Example / key points

```
-- Example when skills is a PostgreSQL text array
SELECT id, full_name, email
FROM candidates
WHERE skills @> ARRAY['python','sql'];
```

67. SQL: Source Effectiveness

Source effectiveness compares where applicants came from with the number eventually hired. The report should show the date range and avoid treating a large applicant count as proof of quality by itself. Consider calculating conversion rates as a separate metric.

Example / key points

```
SELECT c.source,
       COUNT(*) AS applicants,
       COUNT(*) FILTER (WHERE a.stage = 'hired') AS hires
FROM applications a
JOIN candidates c ON c.id = a.candidate_id
GROUP BY c.source
ORDER BY hires DESC;
```

68. SQL: Indexing Strategy

Indexes speed up some reads but add storage and write overhead. Index fields used in frequent filters, joins and ordering. A composite index can help when a query filters by job_id and stage together. Use EXPLAIN ANALYZE on representative data before adding many indexes.

Example / key points

```
CREATE INDEX idx_jobs_status ON jobs(status);
CREATE INDEX idx_app_job_stage ON applications(job_id, stage);
CREATE INDEX idx_app_applied_at ON applications(applied_at);
Review actual query plans before production changes.
```

69. SQL: Transactions

A transaction groups related database changes so they succeed or fail together. When an application is submitted, candidate creation and application creation should be committed consistently. If a duplicate application is detected, the operation should roll back rather than leave partial records. SQLAlchemy sessions provide transaction support, but the service should explicitly handle exceptions and cleanup.

Example / key points

- Begin a transaction.
- Validate the job and candidate.
- Insert or update the candidate.
- Insert the application.
- Commit on success; roll back on failure.

70. SQL Injection Prevention

SQL injection occurs when untrusted input changes the meaning of a query. Never build SQL by joining raw user text into a query string. Use SQLAlchemy query expressions or bound parameters. Input validation is helpful but is not a replacement for parameterised database access.

Example / key points

- Unsafe pattern: query = '... WHERE name = ' + user_input
- Safer pattern: use a bound parameter or ORM filter.
- Limit permissions of the database account.
- Test search endpoints with unusual input.

71. Python: Virtual Environment

A virtual environment keeps project dependencies isolated from other Python projects. Create it inside the backend folder and activate it before installing dependencies. Record dependencies in requirements.txt so another developer can recreate the environment.

Example / key points

```
python -m venv .venv
Windows: .venv\\Scripts\\activate
macOS/Linux: source .venv/bin/activate
python -m pip install fastapi uvicorn sqlalchemy psycopg[binary] pydantic-settings
pip freeze > requirements.txt
```

72. Python: FastAPI Entry Point

The entry point creates the FastAPI application and registers route modules. Keep the entry point small. Health checks can be used by local scripts or hosting platforms to verify that the process responds. A health endpoint should not expose database credentials or other secret configuration.

Example / key points

```
from fastapi import FastAPI

app = FastAPI(title='Recruitment ATS API', version='1.0')

@app.get('/health')
def health():
    return {'status': 'ok'}
```

73. Python: Pydantic Schema

Pydantic schemas validate incoming values before the service processes them. Define minimum and maximum lengths and use appropriate types. Avoid accepting fields that the client should not control, such as created_by or approval identity; derive those values from the authenticated user.

Example / key points

```
from pydantic import BaseModel, Field

class JobCreate(BaseModel):
    title: str = Field(min_length=3, max_length=150)
    description: str = Field(min_length=10)
    employment_type: str
    openings: int = Field(default=1, ge=1)
```

74. Python: Database Session

A database session manages database interaction for one request. Dependency injection lets FastAPI provide a session and close it after the request. Do not share one session globally across concurrent requests. Configure connection details through environment variables.

Example / key points

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

75. Python: Candidate Model Example

An ORM model maps a Python class to a database table. This example shows a small candidate model; production code should align model definitions with the actual migration schema. Relationships and indexes can be added as requirements become clearer.

Example / key points

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from sqlalchemy import String

class Base(DeclarativeBase):
    pass

class Candidate(Base):
    __tablename__ = 'candidates'
    id: Mapped[int] = mapped_column(primary_key=True)
    full_name: Mapped[str] = mapped_column(String(120))
    email: Mapped[str] = mapped_column(String(255), unique=True)
```

76. Python: Stage Transition Rules

Centralising workflow rules prevents different API endpoints from implementing inconsistent transitions. Keep the allowed transitions in one place and return a client error when a transition is invalid. The database should be updated only after the transition is verified.

Example / key points

```
ALLOWED = {
    'applied': {'screening', 'rejected'},
    'screening': {'interview', 'rejected'},
    'interview': {'offer', 'rejected'},
    'offer': {'hired', 'rejected'},
    'hired': set(),
    'rejected': {'screening'},
}
```

77. Python: API Error Handling

Errors should help the caller understand what happened without revealing internal implementation details. Use 401 for unauthenticated requests, 403 for forbidden actions, 404 for missing resources, 409 for conflicts and 422 for invalid request data where appropriate. Log detailed diagnostics on the server while returning a safe message to the client.

Example / key points

```
from fastapi import HTTPException

def require_application(application):
    if application is None:
        raise HTTPException(status_code=404,
                            detail='Application not found')
    return application
```

78. JavaScript: Validate an Email Field

Client-side checks provide immediate feedback, but email validation should remain pragmatic. A browser email input checks basic syntax; the backend should normalise the address and apply required business rules. Do not use a complicated regular expression as the only validation mechanism.

Example / key points

```
function validateEmail(value) {
  const input = value.trim();
  return input.length > 0 &&
    /^[^\\s@]+@[^\\s@]+\\.[^\\s@]+$/.test(input);
}
```

79. JavaScript: Submit a Form

Use FormData when a form includes a file. The browser will set the correct multipart boundary, so do not manually set the Content-Type header for a FormData request. Show a pending state while the request runs and display an error if the server rejects the submission.

Example / key points

```
async function submitApplication(formElement) {
  const data = new FormData(formElement);
  const response = await fetch('/api/v1/public/jobs/1/apply', {
    method: 'POST', body: data
  });
  if (!response.ok) throw new Error('Application failed');
  return response.json();
}
```

80. JavaScript: Render a Job Card

A job card presents the most important vacancy information and a clear next action. In React, use a component with props rather than generating raw HTML strings. Include the job title, location, employment type and an Apply link. Avoid showing closed vacancies as open opportunities.

Example / key points

```
function JobCard({ job }) {
  return (
    <article className='job-card'>
      <h2>{job.title}</h2>
      <p>{job.location} · {job.employment_type}</p>
      <a href={`/jobs/${job.id}/apply`}>Apply now</a>
    </article>
  );
}
```

81. JavaScript: Fetch Published Jobs

The careers page can load published vacancies from a public endpoint. The interface should distinguish loading, empty and error states. Search and filter controls should update the displayed list without losing keyboard accessibility.

Example / key points

```
async function loadJobs() {
  const response = await fetch('/api/v1/public/jobs');
  if (!response.ok) {
    throw new Error('Could not load jobs');
  }
  return response.json();
}
```

82. JavaScript: Display Status Safely

Application status should be displayed with both a text label and optional visual styling. Do not rely on colour alone. Use a mapping from known stage values to labels, and provide a fallback for unexpected data.

Example / key points

```
const stageLabels = {
  applied: 'Applied', screening: 'Screening',
  interview: 'Interview', offer: 'Offer',
  hired: 'Hired', rejected: 'Rejected'
};

function stageLabel(stage) {
  return stageLabels[stage] ?? 'Unknown status';
}
```

83. HTML: Application Form

The form should be straightforward and accessible. Labels must remain visible rather than relying only on placeholder text. The resume field should state accepted formats and maximum size. A server response should be displayed in a status element with clear wording.

Example / key points

```
<form id='apply-form'>
  <label for='full_name'>Full name</label>
  <input id='full_name' name='full_name' required>
  <label for='email'>Email</label>
  <input id='email' name='email' type='email' required>
  <label for='resume'>Resume (PDF/DOCX)</label>
  <input id='resume' name='resume' type='file' accept='.pdf,.docx' required>
  <button type='submit'>Submit application</button>
</form>
```

84. CSS: Job Cards

A consistent card design helps candidates compare openings quickly. Use adequate whitespace, a clear title, short metadata and one primary action. The card should remain readable when the title wraps to two or three lines.

Example / key points

```
.job-card {
  border: 1px solid #d8dee8;
  border-radius: 12px;
  padding: 1rem;
  background: #fff;
  box-shadow: 0 2px 8px rgba(0,0,0,.05);
}
.job-card h2 { margin-top: 0; }
```

85. API Documentation

Document every endpoint with its method, path, purpose, access requirement, request fields, response shape and possible errors. Keep examples free of real personal data. FastAPI's OpenAPI page can help developers inspect the current implementation, but the project report should still explain the most important workflows in plain language.

Example / key points

- Document authentication requirements.
- State required and optional fields.
- Describe pagination and filter parameters.
- List likely status codes.
- Keep API examples synchronised with code.

86. Git and GitHub Workflow

Version control records changes and makes collaboration safer. Commit small, logical changes with descriptive messages. Keep secrets, database dumps and real candidate resumes out of Git. A README should explain installation, environment variables, database setup, test commands and how to run the frontend and backend.

Example / key points

- git init
- git add .
- git commit -m 'Initial ATS project structure'
- Create a GitHub repository and add its remote.
- Push the project after checking that secrets are excluded.

87. Environment Variables

Environment variables separate deployment-specific settings from code. Typical settings include the database URL, secret key, allowed frontend origins, upload directory and mail server credentials. Keep a .env.example file with placeholder values, but do not commit the real .env file.

Example / key points

- DATABASE_URL=postgresql+psycopg://user:password@localhost:5432/ats
- SECRET_KEY=replace-with-a-long-random-secret
- UPLOAD_DIR=uploads
- MAX_RESUME_MB=5
- Never use example credentials in production.

88. Sample Test Plan

A test plan states what will be tested, the test environment, entry criteria and completion criteria. Use synthetic candidates and fictional vacancies. Record actual results after running the application; do not mark tests as passed before execution. Keep screenshots or logs for important failures and fixes.

Example / key points

- Environment: local frontend, API and test database.
- Data: fictional candidate and job records.
- Entry: migrations applied and test configuration loaded.
- Exit: critical flows pass and known defects are documented.

89. Sample Test Case: Duplicate Application

This test verifies that a candidate cannot submit a second application for the same vacancy. Create a published job and a candidate, submit the application once, then repeat the request with the same candidate email and job. The API should return a conflict or another clearly documented duplicate response.

Example / key points

- Test data: one published job and one candidate email.
- First submission: expected success.
- Second submission: expected duplicate conflict.
- Database: only one application for the job-candidate pair.

90. Sample Test Case: Stage Transition

This test verifies that application workflow rules are enforced on the server. Submit an application and attempt a valid transition from Applied to Screening. Then attempt an invalid direct transition from Applied to Hired. The valid request should update stage and history; the invalid request should not modify either.

Example / key points

- Applied → Screening: expected success.
- Applied → Hired: expected rejection.
- Check application.stage after each request.
- Check stage-history rows for consistency.

91. Sample Test Case: File Validation

File validation should reject unsupported formats and oversized uploads. Test a valid small PDF, a DOCX, an executable file renamed with a PDF extension, an empty file and a file above the configured limit. Extension checking alone is insufficient for production; MIME and file-content checks should be considered.

Example / key points

- Valid PDF under limit: accepted.
- Unsupported extension: rejected.
- Oversized file: rejected.
- Suspicious file content: quarantined or rejected.
- Ensure temporary files are cleaned up after failure.

92. Privacy and Data Retention

Candidate profiles and resumes contain personal information. Collect only information needed for recruitment, explain how it will be used and limit staff access. Establish a retention policy and a process for handling deletion or correction requests. Legal obligations vary by location and organisation, so the project report should describe a proposed policy rather than claim universal compliance.

Example / key points

- Minimise collected personal data.
- Restrict resume and feedback access.
- Avoid putting personal information in application logs.
- Define retention and deletion procedures.
- Use synthetic data in demonstrations.

93. Accessibility

Accessibility helps users with different abilities complete recruitment tasks. Use semantic HTML, associated labels, keyboard navigation, visible focus, sufficient contrast and meaningful error messages. For file upload errors, explain both the problem and the expected file format. Test the form without a mouse and inspect it at different zoom levels.

Example / key points

- Use headings in logical order.
- Associate labels with inputs.
- Provide text labels for status colours.
- Keep focus visible.
- Announce success and validation feedback accessibly.

94. Performance Optimisation

Performance should be measured rather than guessed. Use pagination for application lists, select only needed columns and add indexes based on query plans. On the frontend, avoid unnecessary rerenders and large initial bundles. Caching may help for public job listings, but sensitive candidate data requires careful cache controls.

Example / key points

- Paginate large lists.
- Measure slow SQL queries.
- Avoid fetching full resumes for list screens.
- Cache only data that is safe to cache.
- Use production builds for frontend performance checks.

95. Backup and Recovery

A backup is useful only if it can be restored. Define a backup schedule, retention period and recovery procedure. Store backup copies separately from the primary database and restrict access. Periodically test restoring a copy into a non-production environment.

Example / key points

- Back up PostgreSQL on a defined schedule.
- Encrypt backup storage where appropriate.
- Keep access permissions limited.
- Document restoration commands and dependencies.
- Record the date of the last successful restore test.

96. Deployment Checklist

Before deployment, verify that production configuration is complete and no development secrets remain. Run tests and migrations, configure HTTPS and CORS, confirm database backups and check upload permissions. Test the public application flow and the staff-only routes after release.

Example / key points

- Production secret key is configured.
- CORS allows only expected frontend origins.
- Database migration is reviewed.
- HTTPS and secure headers are enabled.
- Resume storage is private.
- Health checks and logs are available.

97. Demonstration Script

A project demonstration should follow one coherent story. Start with the public careers page, open a vacancy and submit a fictional candidate application. Sign in as a recruiter, locate the new application and move it to Screening. Schedule a mock interview, add feedback and show a pipeline report. Explain which features are implemented and which are proposed future work.

Example / key points

- 1. Browse jobs.
- 2. Submit a fictional application.
- 3. Open recruiter dashboard.
- 4. Change application stage.
- 5. Show interview and feedback modules.
- 6. Explain limitations and future scope.

98. Viva Questions: Project Basics

Prepare to explain the problem, users, modules and technology stack in simple language. Answer with a short definition, then connect the answer to one part of the implementation. If a feature is only planned, say so clearly instead of claiming it is complete.

Example / key points

- What is an ATS?
- Why did you choose React?
- Why use FastAPI?
- Why is PostgreSQL suitable?
- Who are the users of the system?
- What is the main benefit of this project?

99. Viva Questions: Database

Database questions often focus on keys, relationships, constraints and query design. Explain why candidates and applications are separate entities, how foreign keys maintain referential integrity and how a unique constraint prevents duplicate applications. Be ready to describe one report query using GROUP BY.

Example / key points

- What is a primary key?
- What is a foreign key?
- Why normalise a database?
- What is an index?
- How do you prevent duplicate applications?
- What does GROUP BY do?

100. Viva Questions: Backend

The backend handles trusted business rules and data access. Explain how FastAPI maps routes to functions, how Pydantic validates request bodies and how a service layer checks stage transitions. Discuss HTTP status codes and why database credentials should be configured outside source code.

Example / key points

- What is a REST API?
- What is dependency injection?
- Why use Pydantic?
- How are errors returned?
- How do you protect an endpoint?
- What is an ORM?

101. Viva Questions: Frontend

The frontend presents information and captures user actions. Explain the role of HTML, CSS and JavaScript, and how React components reuse interface patterns. Describe loading, success and error states and why browser validation is not enough to secure an application.

Example / key points

- What is a React component?
- What are props and state?
- How does the frontend call the API?
- What is responsive design?
- Why use labels on form fields?
- What is client-side validation?

102. Viva Questions: Security

Security is a core part of recruitment because candidate records are private. Explain password hashing, role-based access, server-side validation and safe resume handling. Avoid saying that JWT alone makes an application secure; security also depends on token storage, expiry, transport security and endpoint permissions.

Example / key points

- What is authentication?
- What is authorisation?
- What is SQL injection?
- Why hash passwords?
- How do you validate uploaded files?
- Why use HTTPS?

103. Project Schedule

A schedule divides work into achievable tasks and allows progress to be tracked. The dates should be adjusted to the student's actual project timeline. Add a weekly review and leave time for integration, testing and report formatting rather than allocating all available time to coding.

Example / key points

- Week 1: requirements and design.
- Week 2: schema and backend setup.
- Week 3: jobs and candidate application.
- Week 4: recruiter pipeline and roles.
- Week 5: interviews, reports and testing.
- Week 6: documentation and demonstration.

104. Cost Estimation

The academic prototype can be developed with free or open-source tools. A public deployment may require hosting, a domain, email delivery and object storage. The report should use current vendor prices only if a real deployment budget is requested; for a student project, list cost categories and mark estimates as illustrative.

Example / key points

- Development tools: typically no licence cost for the listed open-source stack.
- Hosting: depends on provider and usage.
- Domain: optional for local demonstration.
- Email and file storage: depends on volume.
- Maintenance: developer time and backups.

105. Expected Outcomes

The expected outcome is a working prototype that demonstrates the recruitment lifecycle with a coherent data model and role-aware interface. Success should be evaluated through test results and a demonstration, not through claims alone. The system should make it possible to locate a vacancy, submit an application and review its progress in a controlled workflow.

Example / key points

- Centralised recruitment records.
- Clear application stage history.
- Structured interview feedback.
- Basic hiring metrics.
- Documented setup and test procedure.

106. References and Learning Resources

The final report should list official documentation and any tutorials consulted during development. Cite the versions actually used. The following official resources are appropriate starting points; check their current content before submission.

Example / key points

- React documentation: https://react.dev/
- FastAPI documentation: https://fastapi.tiangolo.com/
- PostgreSQL documentation: https://www.postgresql.org/docs/
- SQLAlchemy documentation: https://docs.sqlalchemy.org/
- Python documentation: https://docs.python.org/3/

107. Module Detail 107

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

108. Implementation Notes 108

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

109. Design Decision 109

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

110. Operational Scenario 110

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

111. Quality Checklist 111

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

112. Module Detail 112

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

113. Implementation Notes 113

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

114. Design Decision 114

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

115. Operational Scenario 115

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

116. Quality Checklist 116

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

117. Module Detail 117

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

118. Implementation Notes 118

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

119. Design Decision 119

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

120. Operational Scenario 120

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

121. Quality Checklist 121

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

122. Module Detail 122

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

123. Implementation Notes 123

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

124. Design Decision 124

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

125. Operational Scenario 125

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

126. Quality Checklist 126

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

127. Module Detail 127

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

128. Implementation Notes 128

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

129. Design Decision 129

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

130. Operational Scenario 130

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

131. Quality Checklist 131

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

132. Module Detail 132

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

133. Implementation Notes 133

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

134. Design Decision 134

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

135. Operational Scenario 135

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

136. Quality Checklist 136

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

137. Module Detail 137

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

138. Implementation Notes 138

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

139. Design Decision 139

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

140. Operational Scenario 140

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

141. Quality Checklist 141

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

142. Module Detail 142

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

143. Implementation Notes 143

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

144. Design Decision 144

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

145. Operational Scenario 145

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

146. Quality Checklist 146

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

147. Module Detail 147

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

148. Implementation Notes 148

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

149. Design Decision 149

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

150. Operational Scenario 150

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

151. Quality Checklist 151

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

152. Module Detail 152

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

153. Implementation Notes 153

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

154. Design Decision 154

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

155. Operational Scenario 155

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

156. Quality Checklist 156

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

157. Module Detail 157

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

158. Implementation Notes 158

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

159. Design Decision 159

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

160. Operational Scenario 160

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

161. Quality Checklist 161

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

162. Module Detail 162

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

163. Implementation Notes 163

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

164. Design Decision 164

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

165. Operational Scenario 165

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

166. Quality Checklist 166

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

167. Module Detail 167

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

168. Implementation Notes 168

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

169. Design Decision 169

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

170. Operational Scenario 170

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

171. Quality Checklist 171

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

172. Module Detail 172

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

173. Implementation Notes 173

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

174. Design Decision 174

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

175. Operational Scenario 175

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

176. Quality Checklist 176

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

177. Module Detail 177

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

178. Implementation Notes 178

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

179. Design Decision 179

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

180. Operational Scenario 180

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

181. Quality Checklist 181

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

182. Module Detail 182

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

183. Implementation Notes 183

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

184. Design Decision 184

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

185. Operational Scenario 185

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

186. Quality Checklist 186

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

187. Module Detail 187

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

188. Implementation Notes 188

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

189. Design Decision 189

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

190. Operational Scenario 190

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

191. Quality Checklist 191

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

192. Module Detail 192

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

193. Implementation Notes 193

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

194. Design Decision 194

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

195. Operational Scenario 195

Describe a realistic but fictional recruitment scenario and explain how the ATS handles it. Use made-up names and contact details. Walk through the user action, API validation, database change and resulting screen so the design is connected to actual usage.

Example / key points

- Fictional user starts the workflow.
- Frontend validates required input.
- Backend enforces permissions and rules.
- Database records the outcome.
- UI shows a safe confirmation.
- ATS application point: review the job posting workflow and document the result.

196. Quality Checklist 196

A quality checklist helps the developer review a module before considering it complete. Confirm that the interface is accessible, the API handles errors, the database constraints are correct and tests cover important edge cases. Completion should be based on evidence rather than assumptions.

Example / key points

- Readable code and meaningful names
- Server-side validation
- Permission checks
- Tests for success and failure
- Updated documentation
- ATS application point: review the candidate profile workflow and document the result.

197. Module Detail 197

Explain how the module receives input, validates it, calls the service layer and returns a result. Document normal behaviour and at least one failure path. A module should have a clear boundary and should not duplicate rules already enforced elsewhere.

Example / key points

- Inputs and validation
- Business rules
- Database interaction
- Output and error cases
- ATS application point: review the application stage workflow and document the result.

198. Implementation Notes 198

For this topic, describe a practical implementation sequence. Start with the smallest useful version, verify it with sample data, then add validation and access control. Record assumptions so that another student can reproduce the same result.

Example / key points

- Create the module skeleton.
- Implement one happy path.
- Add negative-path handling.
- Write a test before extending the feature.
- Update the README and report.
- ATS application point: review the interview schedule workflow and document the result.

199. Design Decision 199

A design decision should state the problem, alternatives, selected option and reason. For example, a relational database is appropriate for jobs and applications because they have clear relationships and integrity requirements. Revisit the decision if real usage shows a different need.

Example / key points

- Problem being solved
- Options considered
- Selected approach
- Trade-offs and future review
- ATS application point: review the offer record workflow and document the result.

200. Part B: Implementation Code and Output

Part A of this report (pages 1 to 200) describes the Recruitment Applicant Tracking System (ATS) design using HTML, CSS, JavaScript, React, Python/FastAPI and PostgreSQL. Part B adds working implementation material: a **MySQL** database, **Python** and **Java** backend logic, **JavaScript** validation and a complete **HTML** careers page, each with the code and its output. MySQL is offered here as an alternative to the PostgreSQL used in Part A, and Java as an alternative to the Python backend.

What was executed to produce the outputs

Component	Status	How the output was produced
Python core logic (ats_core.py)	Executed	Run with Python 3 on an in-memory SQLite database.
Java service (AtsDemo.java)	Executed	Run with OpenJDK 21 (java AtsDemo.java).
JavaScript validation	Executed	Run with Node.js 22 (node validation_test.js).
HTML careers page	Executed	Opened in headless Chromium; screenshots taken after typing into the form.
MySQL schema and queries	Not run on MySQL	No MySQL server was available. The report queries were run in SQLite with the same data; MySQL-only syntax (ENUM, FIELD, DATEDIFF) is used in the .sql files and should be tested on your server.
FastAPI app (main.py)	Syntax-checked only	Needs a running MySQL server and the fastapi and mysql-connector packages.
Java JDBC DAO (ApplicationDao.java)	Compiled only	Needs a running MySQL server and the MySQL Connector/J driver.

Contents of Part B

200. Part B: Implementation Code and Output	201
201. Project Folder Structure and Setup	202
202. MySQL: Database Schema	203
203. MySQL: Table Relationships and Constraints	205
204. MySQL: Seed Data, Reports and Transaction	206
205. Python: Business Logic (ats_core.py)	207
206. Python: Output of ats_core.py	209
207. Python: FastAPI and MySQL REST API (main.py)	210
208. API Request and Response Examples	212
209. Java: Stage Rules Service (AtsDemo.java)	213
210. Java: Output of AtsDemo.java	215
211. Java: JDBC Data-Access Class for MySQL	216
212. JavaScript: Form Validation (validation.js)	218
213. JavaScript: Output of the Validation Tests	219
214. HTML, CSS and JavaScript: Careers Page	220
215. HTML Page: Output Screenshots	223
216. Test Cases Observed	224
217. Limitations and Next Steps for Part B	225
218. Conclusion of Part B	226

201. Project Folder Structure and Setup

The code in this part is organised so each technology sits in its own folder. The same folder layout is assumed by the run commands below.

```
ats/
  sql/
    schema_mysql.sql        tables, keys and indexes
    reports_mysql.sql       seed data and report queries
  python/
    ats_core.py             business rules + reports (runnable demo)
    main.py                 FastAPI + MySQL REST API
  java/
    AtsDemo.java            stage rules service (runnable demo)
    ApplicationDao.java     JDBC data-access class for MySQL
  js/
    validation.js           form validation (browser and Node)
    validation_test.js      Node test runner
  html/
    careers.html            careers page with job cards and apply form
```

Run commands

```
# 1. Database (MySQL 8)
mysql -u root -p < sql/schema_mysql.sql
mysql -u root -p ats_db < sql/reports_mysql.sql

# 2. Python demo (no database server needed)
python3 python/ats_core.py

# 3. Python API
pip install fastapi uvicorn mysql-connector-python
export DB_PASSWORD='your-password'
uvicorn main:app --reload          # then open http://127.0.0.1:8000/docs

# 4. Java demo
java java/AtsDemo.java             # Java 17 or newer

# 5. JavaScript tests
node js/validation_test.js

# 6. Front end: open html/careers.html in a browser
```

The careers page falls back to three sample jobs when /api/jobs is not reachable, so it can be viewed without the backend.

202. MySQL: Database Schema

The schema has eight tables in third normal form. InnoDB provides foreign keys and transactions. The unique key `uq_job_candidate` stops the same candidate applying to the same job twice, and `application_stage_history` keeps an audit trail of every stage change.

File: sql/schema_mysql.sql

```
CREATE DATABASE IF NOT EXISTS ats_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE ats_db;

CREATE TABLE roles (
  id   TINYINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(30) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE users (
  id            INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  full_name     VARCHAR(120) NOT NULL,
  email         VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role_id       TINYINT UNSIGNED NOT NULL,
  is_active     TINYINT(1) NOT NULL DEFAULT 1,
  created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_users_role FOREIGN KEY (role_id) REFERENCES roles(id)
) ENGINE=InnoDB;

CREATE TABLE jobs (
  id              INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  title           VARCHAR(150) NOT NULL,
  department      VARCHAR(80)  NOT NULL,
  location        VARCHAR(100) NOT NULL,
  employment_type ENUM('Full-time','Part-time','Contract','Internship')
                  NOT NULL DEFAULT 'Full-time',
  description     TEXT NOT NULL,
  status          ENUM('Draft','Published','Closed') NOT NULL DEFAULT 'Draft',
  posted_by       INT UNSIGNED NOT NULL,
  closes_on       DATE NULL,
  created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_jobs_user FOREIGN KEY (posted_by) REFERENCES users(id),
  INDEX idx_jobs_status (status)
) ENGINE=InnoDB;

CREATE TABLE candidates (
  id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  full_name   VARCHAR(120) NOT NULL,
  email       VARCHAR(255) NOT NULL UNIQUE,
  phone       VARCHAR(20),
  skills      VARCHAR(500) NOT NULL DEFAULT '',
  source      ENUM('Careers Page','Referral','LinkedIn','Job Board','Campus')
              NOT NULL DEFAULT 'Careers Page',
  resume_path VARCHAR(255),
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FULLTEXT KEY ft_candidate_skills (skills)
) ENGINE=InnoDB;

CREATE TABLE applications (
  id           INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  job_id       INT UNSIGNED NOT NULL,
  candidate_id INT UNSIGNED NOT NULL,
  stage        ENUM('Applied','Screening','Interview','Offer','Hired','Rejected')
               NOT NULL DEFAULT 'Applied',
  applied_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
               ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_app_job  FOREIGN KEY (job_id)       REFERENCES jobs(id),
  CONSTRAINT fk_app_cand FOREIGN KEY (candidate_id) REFERENCES candidates(id),
  UNIQUE KEY uq_job_candidate (job_id, candidate_id),
  INDEX idx_app_stage (stage)
) ENGINE=InnoDB;

CREATE TABLE application_stage_history (
  id             BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  application_id INT UNSIGNED NOT NULL,
  from_stage     VARCHAR(20) NOT NULL,
  to_stage       VARCHAR(20) NOT NULL,
  changed_by     INT UNSIGNED NULL,
  changed_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  CONSTRAINT fk_hist_app FOREIGN KEY (application_id) REFERENCES applications(id)
) ENGINE=InnoDB;

CREATE TABLE interviews (
  id             INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  application_id INT UNSIGNED NOT NULL,
  interviewer_id INT UNSIGNED NOT NULL,
  scheduled_at   DATETIME NOT NULL,
  mode           ENUM('Online','On-site') NOT NULL DEFAULT 'Online',
  status         ENUM('Scheduled','Completed','Cancelled') NOT NULL DEFAULT 'Scheduled',
  CONSTRAINT fk_int_app  FOREIGN KEY (application_id) REFERENCES applications(id),
  CONSTRAINT fk_int_user FOREIGN KEY (interviewer_id) REFERENCES users(id)
) ENGINE=InnoDB;

CREATE TABLE feedback (
  id             INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  interview_id   INT UNSIGNED NOT NULL UNIQUE,
  rating         TINYINT UNSIGNED NOT NULL,
  recommendation ENUM('Strong Yes','Yes','No','Strong No') NOT NULL,
  comments       TEXT,
  CONSTRAINT chk_rating CHECK (rating BETWEEN 1 AND 5),
  CONSTRAINT fk_fb_int FOREIGN KEY (interview_id) REFERENCES interviews(id)
) ENGINE=InnoDB;
```

203. MySQL: Table Relationships and Constraints

Table	Primary key	Foreign keys / unique rules	Purpose
roles	id	name UNIQUE	Admin, Recruiter, Hiring Manager, Interviewer
users	id	role_id -> roles; email UNIQUE	Staff accounts (password stored as a hash)
jobs	id	posted_by -> users	Vacancies with Draft / Published / Closed status
candidates	id	email UNIQUE	One record per person, reused across jobs
applications	id	job_id -> jobs; candidate_id -> candidates; UNIQUE(job_id, candidate_id)	A candidate's application and current stage
application_stage_history	id	application_id -> applications	Audit trail of stage changes
interviews	id	application_id -> applications; interviewer_id -> users	Scheduled interview slots
feedback	id	interview_id -> interviews, UNIQUE; rating CHECK 1 to 5	One feedback record per interview

Design notes

- **One candidate, many applications:** candidates and applications are separate tables, so a person who applies to three jobs has one candidate row and three application rows.
- **Enforced in the database:** duplicate applications and invalid ratings are rejected by constraints even if the application code has a bug.
- **Stage rules in code:** the allowed stage order (Applied, Screening, Interview, Offer, Hired, with Rejected possible at any point before Hired) is enforced in the Python and Java services.
- **Indexes:** idx_app_stage speeds up pipeline counts; idx_jobs_status speeds up the public jobs list.

204. MySQL: Seed Data, Reports and Transaction

These statements insert starter data and run the four dashboard reports. Parameter placeholders (?) are bound by the driver, never built by string concatenation, which prevents SQL injection.

File: sql/reports_mysql.sql

```
-- Seed data
INSERT INTO roles (name) VALUES ('Admin'),('Recruiter'),('Hiring Manager'),('Interviewer');

INSERT INTO users (full_name, email, password_hash, role_id)
VALUES ('Asha Rao', 'asha@example.com', '$2b$12$hash-placeholder', 2);

INSERT INTO jobs (title, department, location, employment_type,
                  description, status, posted_by)
VALUES ('Java Developer', 'Engineering', 'Hyderabad', 'Full-time',
        'Build REST services.', 'Published', 1),
       ('Python Backend Engineer', 'Engineering', 'Remote', 'Full-time',
        'FastAPI and MySQL.', 'Published', 1),
       ('UI Developer', 'Design', 'Guntur', 'Contract',
        'HTML, CSS and JavaScript.', 'Published', 1);

-- Report 1: pipeline totals by stage
SELECT stage, COUNT(*) AS total
FROM applications
GROUP BY stage
ORDER BY FIELD(stage,'Applied','Screening','Interview',
                     'Offer','Hired','Rejected');

-- Report 2: candidates by skill (parameterised)
SELECT full_name, email, skills
FROM candidates
WHERE skills LIKE CONCAT('%', ?, '%')
ORDER BY full_name;

-- Report 3: source effectiveness
SELECT c.source,
       COUNT(*)               AS applications,
       SUM(a.stage = 'Hired') AS hired
FROM applications a
JOIN candidates c ON c.id = a.candidate_id
GROUP BY c.source
ORDER BY hired DESC, applications DESC, c.source;

-- Report 4: average time-to-hire in days (MySQL DATEDIFF)
SELECT j.title,
       ROUND(AVG(DATEDIFF(a.updated_at, a.applied_at)), 1) AS avg_days
FROM applications a
JOIN jobs j ON j.id = a.job_id
WHERE a.stage = 'Hired'
GROUP BY j.title;

-- Transaction: move a stage and write history atomically
START TRANSACTION;
UPDATE applications SET stage = 'Screening' WHERE id = 1 AND stage = 'Applied';
INSERT INTO application_stage_history (application_id, from_stage, to_stage, changed_by)
VALUES (1, 'Applied', 'Screening', 1);
COMMIT;
```

205. Python: Business Logic (ats_core.py)

This module holds the rules that matter most: the duplicate-application check, the allowed stage transitions, an atomic stage change with history, and the three reports. It uses SQLite so it runs anywhere; for MySQL, replace the connection and use %s placeholders as shown in main.py.

File: python/ats_core.py

```
"""ATS core logic. Runs on SQLite so it can be demonstrated anywhere.
For MySQL, change connect() and use %s placeholders instead of ?."""
import sqlite3

STAGES = ["Applied", "Screening", "Interview", "Offer", "Hired", "Rejected"]

ALLOWED = {
    "Applied":   {"Screening", "Rejected"},
    "Screening": {"Interview", "Rejected"},
    "Interview": {"Offer", "Rejected"},
    "Offer":     {"Hired", "Rejected"},
    "Hired":     set(),
    "Rejected":  set(),
}


class DuplicateApplication(Exception):
    pass


class InvalidTransition(Exception):
    pass


SCHEMA = """
CREATE TABLE jobs (id INTEGER PRIMARY KEY, title TEXT NOT NULL, status TEXT NOT NULL);
CREATE TABLE candidates (id INTEGER PRIMARY KEY, full_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL, skills TEXT NOT NULL, source TEXT NOT NULL);
CREATE TABLE applications (id INTEGER PRIMARY KEY, job_id INTEGER NOT NULL,
    candidate_id INTEGER NOT NULL, stage TEXT NOT NULL DEFAULT 'Applied',
    UNIQUE (job_id, candidate_id));
CREATE TABLE stage_history (id INTEGER PRIMARY KEY, application_id INTEGER,
    from_stage TEXT, to_stage TEXT);
"""


class AtsService:
    def __init__(self):
        self.db = sqlite3.connect(":memory:")
        self.db.executescript(SCHEMA)

    def add_job(self, title, status="Published"):
        cur = self.db.execute("INSERT INTO jobs (title, status) VALUES (?, ?)", (title, status))
        return cur.lastrowid

    def add_candidate(self, name, email, skills, source):
        cur = self.db.execute(
            "INSERT INTO candidates (full_name, email, skills, source) VALUES (?, ?, ?, ?)",
            (name, email, skills, source))
        return cur.lastrowid

    def apply(self, job_id, candidate_id):
        try:
            cur = self.db.execute(
                "INSERT INTO applications (job_id, candidate_id) VALUES (?, ?)",
                (job_id, candidate_id))
        except sqlite3.IntegrityError:
            raise DuplicateApplication("Candidate already applied to this job")
        return cur.lastrowid

    def move_stage(self, application_id, new_stage):
        row = self.db.execute("SELECT stage FROM applications WHERE id = ?",
                              (application_id,)).fetchone()
        if row is None:
            raise KeyError(f"Application {application_id} not found")
        current = row[0]
        if new_stage not in ALLOWED[current]:
            raise InvalidTransition(f"Cannot move from {current} to {new_stage}")
        with self.db:  # one transaction: update + history
            self.db.execute("UPDATE applications SET stage = ? WHERE id = ?",
                            (new_stage, application_id))
```

```
            self.db.execute(
                "INSERT INTO stage_history (application_id, from_stage, to_stage) VALUES (?, ?, ?)",
                (application_id, current, new_stage))

    def pipeline_counts(self):
        rows = dict(self.db.execute(
            "SELECT stage, COUNT(*) FROM applications GROUP BY stage").fetchall())
        return [(s, rows[s]) for s in STAGES if s in rows]

    def find_by_skill(self, skill):
        return self.db.execute(
            "SELECT full_name, email, skills FROM candidates "
            "WHERE skills LIKE ? ORDER BY full_name", (f"%{skill}%",)).fetchall()

    def source_effectiveness(self):
        return self.db.execute(
            "SELECT c.source, COUNT(*) AS applications, "
            "SUM(CASE WHEN a.stage = 'Hired' THEN 1 ELSE 0 END) AS hired "
            "FROM applications a JOIN candidates c ON c.id = a.candidate_id "
            "GROUP BY c.source ORDER BY hired DESC, applications DESC, c.source").fetchall()

    def history(self, application_id):
        return self.db.execute(
            "SELECT from_stage, to_stage FROM stage_history "
            "WHERE application_id = ? ORDER BY id", (application_id,)).fetchall()


def demo():
    ats = AtsService()
    java_job = ats.add_job("Java Developer")
    py_job = ats.add_job("Python Backend Engineer")
    people = [
        ("Ravi Kumar",  "ravi@example.com",  "Java, Spring, MySQL",       "Referral"),
        ("Sneha Reddy", "sneha@example.com", "Python, FastAPI, MySQL",    "Careers Page"),
        ("Arjun Das",   "arjun@example.com", "JavaScript, HTML, CSS",     "LinkedIn"),
        ("Meena Iyer",  "meena@example.com", "Python, Django, SQL",       "Careers Page"),
    ]
    ids = [ats.add_candidate(*p) for p in people]
    a1 = ats.apply(java_job, ids[0]); a2 = ats.apply(py_job, ids[1])
    a3 = ats.apply(py_job, ids[2]);   a4 = ats.apply(py_job, ids[3])

    print("== Stage movement ==")
    for stage in ["Screening", "Interview", "Offer", "Hired"]:
        ats.move_stage(a2, stage)
    ats.move_stage(a1, "Screening")
    ats.move_stage(a3, "Rejected")
    print("Application", a2, "history:", " -> ".join(
        [ats.history(a2)[0][0]] + [t for _, t in ats.history(a2)]))

    print("\n== Rule checks ==")
    for label, action in [
        ("Duplicate application", lambda: ats.apply(java_job, ids[0])),
        ("Applied -> Hired (skip)", lambda: ats.move_stage(a4, "Hired")),
        ("Hired -> Screening (reopen)", lambda: ats.move_stage(a2, "Screening")),
    ]:
        try:
            action()
            print(f"{label}: ALLOWED (unexpected)")
        except (DuplicateApplication, InvalidTransition) as exc:
            print(f"{label}: blocked - {exc}")

    print("\n== Pipeline totals ==")
    for stage, total in ats.pipeline_counts():
        print(f"{stage:<10} {total}")

    print("\n== Candidates with skill 'Python' ==")
    for name, email, skills in ats.find_by_skill("Python"):
        print(f"{name:<12} {email:<18} {skills}")

    print("\n== Source effectiveness ==")
    print(f"{'Source':<14}{'Applications':>13}{'Hired':>7}")
    for source, apps, hired in ats.source_effectiveness():
        print(f"{source:<14}{apps:>13}{hired:>7}")


if __name__ == "__main__":
    demo()
```

206. Python: Output of ats_core.py

The demo adds two jobs and four candidates, moves applications through the pipeline, then tries three actions that must be blocked.

Command: python3 python/ats_core.py

```
== Stage movement ==
Application 2 history: Applied -> Screening -> Interview -> Offer -> Hired

== Rule checks ==
Duplicate application: blocked - Candidate already applied to this job
Applied -> Hired (skip): blocked - Cannot move from Applied to Hired
Hired -> Screening (reopen): blocked - Cannot move from Hired to Screening

== Pipeline totals ==
Applied    1
Screening  1
Hired      1
Rejected   1

== Candidates with skill 'Python' ==
Meena Iyer   meena@example.com  Python, Django, SQL
Sneha Reddy  sneha@example.com  Python, FastAPI, MySQL

== Source effectiveness ==
Source         Applications  Hired
Careers Page              2      1
LinkedIn                  1      0
Referral                  1      0
```

Reading the output

- Application 2 moved through every stage in order and ended as Hired.
- All three rule checks were blocked: a duplicate application, skipping a stage (Applied to Hired) and reopening a closed application (Hired to Screening).
- Pipeline totals match the data: one application each in Applied, Screening, Hired and Rejected.
- The Careers Page source produced the only hire, so it ranks first in source effectiveness.

207. Python: FastAPI and MySQL REST API (main.py)

This API exposes three endpoints: list published jobs, submit an application, and change an application's stage. Each query is parameterised, the stage change uses `SELECT ... FOR UPDATE` to avoid two recruiters changing the same record at once, and duplicate applications return HTTP 409.

File: python/main.py

```
"""FastAPI backend for the ATS using MySQL.
Install: pip install fastapi uvicorn mysql-connector-python
Run:     uvicorn main:app --reload
"""
import os
import re

import mysql.connector
from mysql.connector import pooling, errors
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field, field_validator

ALLOWED = {
    "Applied": {"Screening", "Rejected"},
    "Screening": {"Interview", "Rejected"},
    "Interview": {"Offer", "Rejected"},
    "Offer": {"Hired", "Rejected"},
    "Hired": set(),
    "Rejected": set(),
}

pool = pooling.MySQLConnectionPool(
    pool_name="ats_pool",
    pool_size=5,
    host=os.getenv("DB_HOST", "localhost"),
    user=os.getenv("DB_USER", "ats_user"),
    password=os.environ["DB_PASSWORD"],
    database=os.getenv("DB_NAME", "ats_db"),
)

app = FastAPI(title="Recruitment ATS API", version="1.0")
EMAIL_RE = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")


class ApplicationIn(BaseModel):
    job_id: int
    full_name: str = Field(min_length=2, max_length=120)
    email: str
    phone: str | None = None
    skills: str = ""

    @field_validator("email")
    @classmethod
    def check_email(cls, value: str) -> str:
        if not EMAIL_RE.match(value):
            raise ValueError("Enter a valid email address")
        return value.lower()


class StageChange(BaseModel):
    new_stage: str


@app.get("/api/jobs")
def list_jobs():
    conn = pool.get_connection()
    try:
        cur = conn.cursor(dictionary=True)
        cur.execute(
            "SELECT id, title, department, location, employment_type "
            "FROM jobs WHERE status = %s ORDER BY created_at DESC", ("Published",))
        return cur.fetchall()
    finally:
        conn.close()


@app.post("/api/applications", status_code=201)
def create_application(data: ApplicationIn):
    conn = pool.get_connection()
    try:
        cur = conn.cursor()
```

```
        cur.execute(
            "INSERT INTO candidates (full_name, email, phone, skills) "
            "VALUES (%s, %s, %s, %s) "
            "ON DUPLICATE KEY UPDATE full_name = VALUES(full_name), "
            "id = LAST_INSERT_ID(id)",
            (data.full_name, data.email, data.phone, data.skills))
        candidate_id = cur.lastrowid
        try:
            cur.execute("INSERT INTO applications (job_id, candidate_id) VALUES (%s, %s)",
                        (data.job_id, candidate_id))
        except errors.IntegrityError:
            conn.rollback()
            raise HTTPException(status_code=409,
                                detail="You have already applied for this job")
        conn.commit()
        return {"application_id": cur.lastrowid, "stage": "Applied"}
    finally:
        conn.close()


@app.patch("/api/applications/{application_id}/stage")
def change_stage(application_id: int, body: StageChange):
    conn = pool.get_connection()
    try:
        cur = conn.cursor()
        cur.execute("SELECT stage FROM applications WHERE id = %s FOR UPDATE",
                    (application_id,))
        row = cur.fetchone()
        if row is None:
            raise HTTPException(status_code=404, detail="Application not found")
        current = row[0]
        if body.new_stage not in ALLOWED.get(current, set()):
            raise HTTPException(status_code=400,
                                detail=f"Cannot move from {current} to {body.new_stage}")
        cur.execute("UPDATE applications SET stage = %s WHERE id = %s",
                    (body.new_stage, application_id))
        cur.execute("INSERT INTO application_stage_history "
                    "(application_id, from_stage, to_stage) VALUES (%s, %s, %s)",
                    (application_id, current, body.new_stage))
        conn.commit()
        return {"application_id": application_id, "from": current, "to": body.new_stage}
    finally:
        conn.close()
```

Status: syntax-checked only. Running it needs MySQL, the packages listed in the file header, and the DB_PASSWORD environment variable.

208. API Request and Response Examples

These examples show what the endpoints in main.py are designed to accept and return. They are illustrative and were not captured from a live server.

```
GET /api/jobs
200 OK
[
  {"id": 1, "title": "Java Developer", "department": "Engineering",
   "location": "Hyderabad", "employment_type": "Full-time"}
]

POST /api/applications
{"job_id": 1, "full_name": "Ravi Kumar", "email": "ravi@example.com",
 "phone": "9876543210", "skills": "Java, Spring, MySQL"}
201 Created
{"application_id": 12, "stage": "Applied"}

POST /api/applications              (same candidate, same job)
409 Conflict
{"detail": "You have already applied for this job"}

PATCH /api/applications/12/stage
{"new_stage": "Screening"}
200 OK
{"application_id": 12, "from": "Applied", "to": "Screening"}

PATCH /api/applications/12/stage
{"new_stage": "Hired"}
400 Bad Request
{"detail": "Cannot move from Screening to Hired"}
```

Status	Meaning in this API
200 / 201	Request succeeded / application created
400	Stage change not allowed by the pipeline rules
404	Application id does not exist
409	Candidate has already applied to this job
422	Validation failed (for example, malformed email); returned automatically by FastAPI

209. Java: Stage Rules Service (AtsDemo.java)

The Java version models the pipeline as an enum. Each stage lists the stages it may move to, so an invalid move is rejected in one place. The service also blocks duplicate applications and reports pipeline totals.

File: java/AtsDemo.java

```
import java.util.*;

public class AtsDemo {

    enum Stage {
        APPLIED, SCREENING, INTERVIEW, OFFER, HIRED, REJECTED;

        Set<Stage> next() {
            return switch (this) {
                case APPLIED   -> EnumSet.of(SCREENING, REJECTED);
                case SCREENING -> EnumSet.of(INTERVIEW, REJECTED);
                case INTERVIEW -> EnumSet.of(OFFER, REJECTED);
                case OFFER     -> EnumSet.of(HIRED, REJECTED);
                case HIRED, REJECTED -> EnumSet.noneOf(Stage.class);
            };
        }

        boolean canMoveTo(Stage target) {
            return next().contains(target);
        }
    }

    static class Application {
        final int id, jobId, candidateId;
        Stage stage = Stage.APPLIED;
        final List<String> history = new ArrayList<>();

        Application(int id, int jobId, int candidateId) {
            this.id = id; this.jobId = jobId; this.candidateId = candidateId;
        }
    }

    static class ApplicationService {
        private final Map<Integer, Application> store = new LinkedHashMap<>();
        private final Set<String> pairs = new HashSet<>();
        private int nextId = 1;

        Application apply(int jobId, int candidateId) {
            if (!pairs.add(jobId + ":" + candidateId)) {
                throw new IllegalStateException("Candidate already applied to this job");
            }
            Application app = new Application(nextId++, jobId, candidateId);
            store.put(app.id, app);
            return app;
        }

        void move(int applicationId, Stage target) {
            Application app = store.get(applicationId);
            if (app == null) throw new NoSuchElementException("Application not found");
            if (!app.stage.canMoveTo(target)) {
                throw new IllegalArgumentException(
                        "Cannot move from " + app.stage + " to " + target);
            }
            app.history.add(app.stage + " -> " + target);
            app.stage = target;
        }

        Map<Stage, Long> pipeline() {
            Map<Stage, Long> counts = new EnumMap<>(Stage.class);
            for (Application a : store.values()) counts.merge(a.stage, 1L, Long::sum);
            return counts;
        }
    }

    public static void main(String[] args) {
        ApplicationService service = new ApplicationService();
        Application a1 = service.apply(101, 1);
        Application a2 = service.apply(101, 2);
        Application a3 = service.apply(102, 3);

        service.move(a1.id, Stage.SCREENING);
        service.move(a1.id, Stage.INTERVIEW);
        service.move(a1.id, Stage.OFFER);
```

```
        service.move(a1.id, Stage.HIRED);
        service.move(a2.id, Stage.REJECTED);

        System.out.println("== Application 1 history ==");
        a1.history.forEach(System.out::println);

        System.out.println("\n== Rule checks ==");
        try { service.apply(101, 1); }
        catch (IllegalStateException e) {
            System.out.println("Duplicate: blocked - " + e.getMessage());
        }
        try { service.move(a3.id, Stage.HIRED); }
        catch (IllegalArgumentException e) { System.out.println("Skip stage: blocked - " + e.getMessage()); }
        try { service.move(a1.id, Stage.SCREENING); }
        catch (IllegalArgumentException e) { System.out.println("Reopen: blocked - " + e.getMessage()); }

        System.out.println("\n== Pipeline totals ==");
        service.pipeline().forEach((stage, n) -> System.out.printf("%-10s %d%n", stage, n));
    }
}
```

210. Java: Output of AtsDemo.java

Command: java java/AtsDemo.java

```
== Application 1 history ==
APPLIED -> SCREENING
SCREENING -> INTERVIEW
INTERVIEW -> OFFER
OFFER -> HIRED

== Rule checks ==
Duplicate: blocked - Candidate already applied to this job
Skip stage: blocked - Cannot move from APPLIED to HIRED
Reopen: blocked - Cannot move from HIRED to SCREENING

== Pipeline totals ==
APPLIED    1
HIRED      1
REJECTED   1
```

Reading the output

- Application 1 completed the full path from APPLIED to HIRED.
- The Java rules produce the same three blocks as the Python version: duplicate, skipped stage and reopen.
- Totals show one application each in APPLIED, HIRED and REJECTED.

211. Java: JDBC Data-Access Class for MySQL

This class connects the Java rules to MySQL. It uses PreparedStatement for every query, catches the duplicate-key error to report a repeat application, and wraps the stage update and history insert in one transaction with rollback. The WHERE clause includes the expected current stage, so a stale update affects zero rows and is detected.

File: java/ApplicationDao.java

```
import java.sql.*;
import java.util.*;

/**
 * JDBC data-access class for MySQL.
 * Add the driver: com.mysql:mysql-connector-j (Maven/Gradle).
 * URL example: jdbc:mysql://localhost:3306/ats_db?useSSL=false&serverTimezone=UTC
 */
public class ApplicationDao {
    private final String url, user, password;

    public ApplicationDao(String url, String user, String password) {
        this.url = url; this.user = user; this.password = password;
    }

    private Connection open() throws SQLException {
        return DriverManager.getConnection(url, user, password);
    }

    /** Insert an application; returns the new id, or -1 if it is a duplicate. */
    public long apply(int jobId, int candidateId) throws SQLException {
        String sql = "INSERT INTO applications (job_id, candidate_id) VALUES (?, ?)";
        try (Connection c = open();
             PreparedStatement ps = c.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setInt(1, jobId);
            ps.setInt(2, candidateId);
            ps.executeUpdate();
            try (ResultSet keys = ps.getGeneratedKeys()) {
                return keys.next() ? keys.getLong(1) : -1;
            }
        } catch (SQLIntegrityConstraintViolationException duplicate) {
            return -1;   // uq_job_candidate prevented a second application
        }
    }

    /** Move an application to a new stage and record history in one transaction. */
    public void moveStage(int applicationId, String from, String to, int userId)
            throws SQLException {
        try (Connection c = open()) {
            c.setAutoCommit(false);
            try (PreparedStatement up = c.prepareStatement(
                        "UPDATE applications SET stage = ? WHERE id = ? AND stage = ?");
                 PreparedStatement log = c.prepareStatement(
                        "INSERT INTO application_stage_history "
                      + "(application_id, from_stage, to_stage, changed_by) VALUES (?,?,?,?)")) {
                up.setString(1, to);
                up.setInt(2, applicationId);
                up.setString(3, from);
                if (up.executeUpdate() != 1) {
                    throw new SQLException("Stage changed by someone else; reload and retry");
                }
                log.setInt(1, applicationId);
                log.setString(2, from);
                log.setString(3, to);
                log.setInt(4, userId);
                log.executeUpdate();
                c.commit();
            } catch (SQLException e) {
                c.rollback();
                throw e;
            }
        }
    }

    /** Pipeline totals grouped by stage. */
    public Map<String, Integer> pipelineCounts() throws SQLException {
        Map<String, Integer> result = new LinkedHashMap<>();
        String sql = "SELECT stage, COUNT(*) FROM applications GROUP BY stage";
        try (Connection c = open();
             Statement st = c.createStatement();
             ResultSet rs = st.executeQuery(sql)) {
```

```
            while (rs.next()) result.put(rs.getString(1), rs.getInt(2));
        }
        return result;
    }
}
```

Status: compiled successfully with OpenJDK 21. Running it needs MySQL and Connector/J (com.mysql:mysql-connector-j).

212. JavaScript: Form Validation (validation.js)

Validation runs in the browser to give instant feedback, and the same module runs in Node so it can be tested. Client-side checks improve usability only; the backend repeats the checks because users can bypass the browser.

File: js/validation.js

```
/* validation.js - works in the browser (window.AtsValidation) and in Node */
(function (root) {
  const EMAIL = /^[^@\s]+@[^@\s]+\.[^@\s]+$/;
  const PHONE = /^[0-9+\-\s]{10,15}$/;
  const MAX_RESUME_BYTES = 2 * 1024 * 1024;           // 2 MB
  const ALLOWED_EXT = ["pdf", "doc", "docx"];

  function validateEmail(value) {
    return EMAIL.test(String(value).trim());
  }

  function validateResume(file) {
    if (!file) return "Please attach your resume";
    const ext = file.name.split(".").pop().toLowerCase();
    if (!ALLOWED_EXT.includes(ext)) return "Resume must be PDF, DOC or DOCX";
    if (file.size > MAX_RESUME_BYTES) return "Resume must be 2 MB or smaller";
    return null;
  }

  function validateApplication(data) {
    const errors = {};
    if (!data.fullName || data.fullName.trim().length < 2) {
      errors.fullName = "Enter your full name";
    }
    if (!validateEmail(data.email || "")) {
      errors.email = "Enter a valid email address";
    }
    if (data.phone && !PHONE.test(data.phone)) {
      errors.phone = "Enter a valid phone number";
    }
    const resumeError = validateResume(data.resume);
    if (resumeError) errors.resume = resumeError;
    return errors;
  }

  const api = { validateEmail, validateResume, validateApplication };
  if (typeof module !== "undefined" && module.exports) module.exports = api;
  else root.AtsValidation = api;
})(typeof window !== "undefined" ? window : globalThis);
```

File: js/validation_test.js

```
const { validateEmail, validateApplication } = require("./validation.js");

console.log("== Email checks ==");
["ravi@example.com", "ravi@", "no-at-sign.com", " sneha@mail.in "].forEach((e) =>
  console.log(JSON.stringify(e).padEnd(20), validateEmail(e) ? "valid" : "invalid")
);

console.log("\n== Application: good data ==");
console.log(validateApplication({
  fullName: "Ravi Kumar", email: "ravi@example.com", phone: "9876543210",
  resume: { name: "ravi_cv.pdf", size: 350000 },
}));

console.log("\n== Application: bad data ==");
console.log(validateApplication({
  fullName: "R", email: "ravi@", phone: "123",
  resume: { name: "photo.png", size: 5000000 },
}));

console.log("\n== Oversized PDF ==");
console.log(validateApplication({
  fullName: "Meena Iyer", email: "meena@example.com",
  resume: { name: "big.pdf", size: 3000000 },
}));
```

213. JavaScript: Output of the Validation Tests

Command: node js/validation_test.js

```
== Email checks ==
"ravi@example.com"   valid
"ravi@"              invalid
"no-at-sign.com"     invalid
" sneha@mail.in "    valid

== Application: good data ==
{}

== Application: bad data ==
{
  fullName: 'Enter your full name',
  email: 'Enter a valid email address',
  phone: 'Enter a valid phone number',
  resume: 'Resume must be PDF, DOC or DOCX'
}

== Oversized PDF ==
{ resume: 'Resume must be 2 MB or smaller' }
```

Reading the output

- Addresses without a domain part (ravi@, no-at-sign.com) are rejected; surrounding spaces are trimmed.
- Valid data returns an empty error object.
- Bad data returns one message per failing field, including the wrong file type for the resume.
- A 3 MB PDF is rejected by the 2 MB limit even though its type is allowed.

214. HTML, CSS and JavaScript: Careers Page

A single page lists open jobs as cards and provides the application form. Job cards are built with `textContent` rather than `innerHTML`, so text from the database can never inject markup or scripts. Labels, focus outlines and an aria-live region support accessibility, and the grid collapses to one column on narrow screens.

File: html/careers.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Careers | Recruitment ATS</title>
  <style>
    :root { --brand:#1f4e8c; --bg:#f4f6fa; --card:#fff; --text:#1d2433;
            --muted:#5b6578; --err:#b3261e; --ok:#1b7f3b; }
    * { box-sizing: border-box; }
    body { margin:0; font-family: Arial, Helvetica, sans-serif; background:var(--bg); color:var(--text); }
    header { background:var(--brand); color:#fff; padding:20px 24px; }
    header h1 { margin:0; font-size:22px; }
    header p { margin:4px 0 0; opacity:.85; font-size:14px; }
    main { max-width:960px; margin:24px auto; padding:0 16px; display:grid; gap:24px;
           grid-template-columns: 1fr 1fr; }
    @media (max-width:760px) { main { grid-template-columns:1fr; } }
    h2 { font-size:17px; margin:0 0 12px; color:var(--brand); }
    .job-card { background:var(--card); border:1px solid #dde2ec; border-radius:8px;
                padding:14px 16px; margin-bottom:12px; }
    .job-card h3 { margin:0 0 4px; font-size:16px; }
    .job-card .meta { color:var(--muted); font-size:13px; }
    .job-card button { margin-top:10px; }
    .badge { display:inline-block; background:#e6eefb; color:var(--brand); border-radius:10px;
             padding:2px 9px; font-size:12px; margin-right:6px; }
    form { background:var(--card); border:1px solid #dde2ec; border-radius:8px; padding:16px; }
    label { display:block; font-size:13px; font-weight:bold; margin:12px 0 4px; }
    input, select { width:100%; padding:9px 10px; border:1px solid #b9c1d1;
                    border-radius:6px; font-size:14px; }
    input:focus, select:focus, button:focus { outline:3px solid #9bbcf0; }
    .error { color:var(--err); font-size:12px; min-height:16px; margin-top:3px; }
    button { background:var(--brand); color:#fff; border:0; border-radius:6px; padding:9px 16px;
             font-size:14px; cursor:pointer; }
    button.secondary { background:#fff; color:var(--brand); border:1px solid var(--brand); }
    #status { margin-top:12px; font-size:14px; }
    #status.ok { color:var(--ok); } #status.fail { color:var(--err); }
  </style>
</head>
<body>
  <header>
    <h1>Join Our Team</h1>
    <p>Browse open positions and apply in two minutes.</p>
  </header>

  <main>
    <section aria-labelledby="jobs-title">
      <h2 id="jobs-title">Open Positions</h2>
      <div id="jobs" aria-live="polite">Loading jobs...</div>
    </section>

    <section aria-labelledby="apply-title">
      <h2 id="apply-title">Apply Now</h2>
      <form id="apply-form" novalidate>
        <label for="job">Position</label>
        <select id="job" name="job"></select>

        <label for="fullName">Full name</label>
        <input id="fullName" name="fullName" autocomplete="name">
        <div class="error" id="err-fullName"></div>

        <label for="email">Email</label>
        <input id="email" name="email" type="email" autocomplete="email">
        <div class="error" id="err-email"></div>

        <label for="phone">Phone (optional)</label>
        <input id="phone" name="phone" type="tel" autocomplete="tel">
        <div class="error" id="err-phone"></div>

        <label for="resume">Resume (PDF, DOC, DOCX, max 2 MB)</label>
        <input id="resume" name="resume" type="file" accept=".pdf,.doc,.docx">
        <div class="error" id="err-resume"></div>
```

```
      <button type="submit">Submit application</button>
      <div id="status" role="status"></div>
    </form>
  </section>
</main>

<script src="../js/validation.js"></script>
<script>
  // Sample data is used when the API is not running (for demos and screenshots).
  const SAMPLE_JOBS = [
    { id: 1, title: "Java Developer", department: "Engineering",
      location: "Hyderabad", employment_type: "Full-time" },
    { id: 2, title: "Python Backend Engineer", department: "Engineering",
      location: "Remote", employment_type: "Full-time" },
    { id: 3, title: "UI Developer", department: "Design",
      location: "Guntur", employment_type: "Contract" }
  ];

  async function fetchJobs() {
    try {
      const res = await fetch("/api/jobs");
      if (!res.ok) throw new Error("HTTP " + res.status);
      return await res.json();
    } catch (err) {
      return SAMPLE_JOBS;
    }
  }

  // textContent (not innerHTML) so job data can never inject HTML or scripts.
  function renderJobCard(job, onApply) {
    const card = document.createElement("article");
    card.className = "job-card";

    const title = document.createElement("h3");
    title.textContent = job.title;

    const meta = document.createElement("div");
    meta.className = "meta";
    const type = document.createElement("span");
    type.className = "badge";
    type.textContent = job.employment_type;
    meta.append(type, document.createTextNode(job.department + " | " + job.location));

    const btn = document.createElement("button");
    btn.type = "button";
    btn.className = "secondary";
    btn.textContent = "Apply";
    btn.addEventListener("click", () => onApply(job.id));

    card.append(title, meta, btn);
    return card;
  }

  function showErrors(errors) {
    ["fullName", "email", "phone", "resume"].forEach((field) => {
      document.getElementById("err-" + field).textContent = errors[field] || "";
    });
  }

  async function init() {
    const jobs = await fetchJobs();
    const list = document.getElementById("jobs");
    const select = document.getElementById("job");
    list.textContent = "";
    jobs.forEach((job) => {
      list.append(renderJobCard(job, (id) => {
        select.value = id;
        document.getElementById("fullName").focus();
      }));
      select.add(new Option(job.title, job.id));
    });

    document.getElementById("apply-form").addEventListener("submit", async (event) => {
      event.preventDefault();
      const status = document.getElementById("status");
      const data = {
        fullName: document.getElementById("fullName").value,
        email: document.getElementById("email").value,
        phone: document.getElementById("phone").value,
        resume: document.getElementById("resume").files[0]
      };
```

```
        const errors = AtsValidation.validateApplication(data);
        showErrors(errors);
        if (Object.keys(errors).length) {
          status.className = "fail";
          status.textContent = "Please fix the highlighted fields.";
          return;
        }
        status.className = "ok";
        status.textContent = "Application ready to send to POST /api/applications.";
      });
    }
    init();
  </script>
</body>
</html>
```

215. HTML Page: Output Screenshots

Screenshots were taken from the page above in headless Chromium at 960 px width.

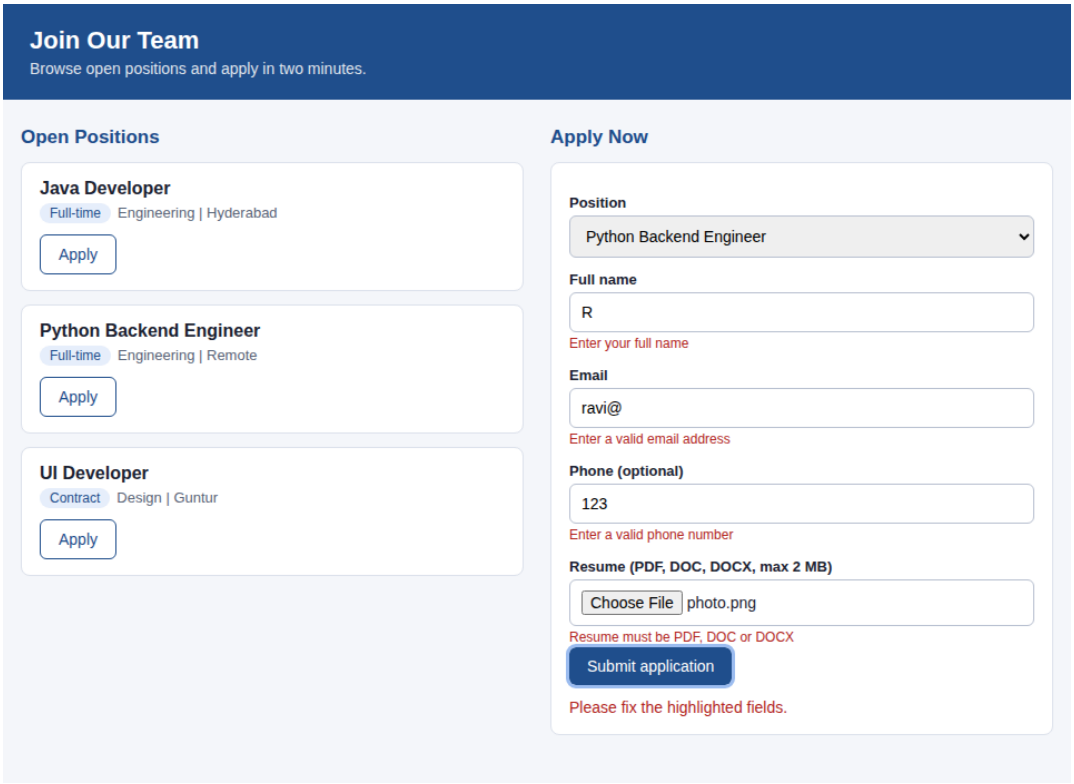


Figure B1. Invalid input (short name, bad email, short phone, PNG resume): each field shows its own message.

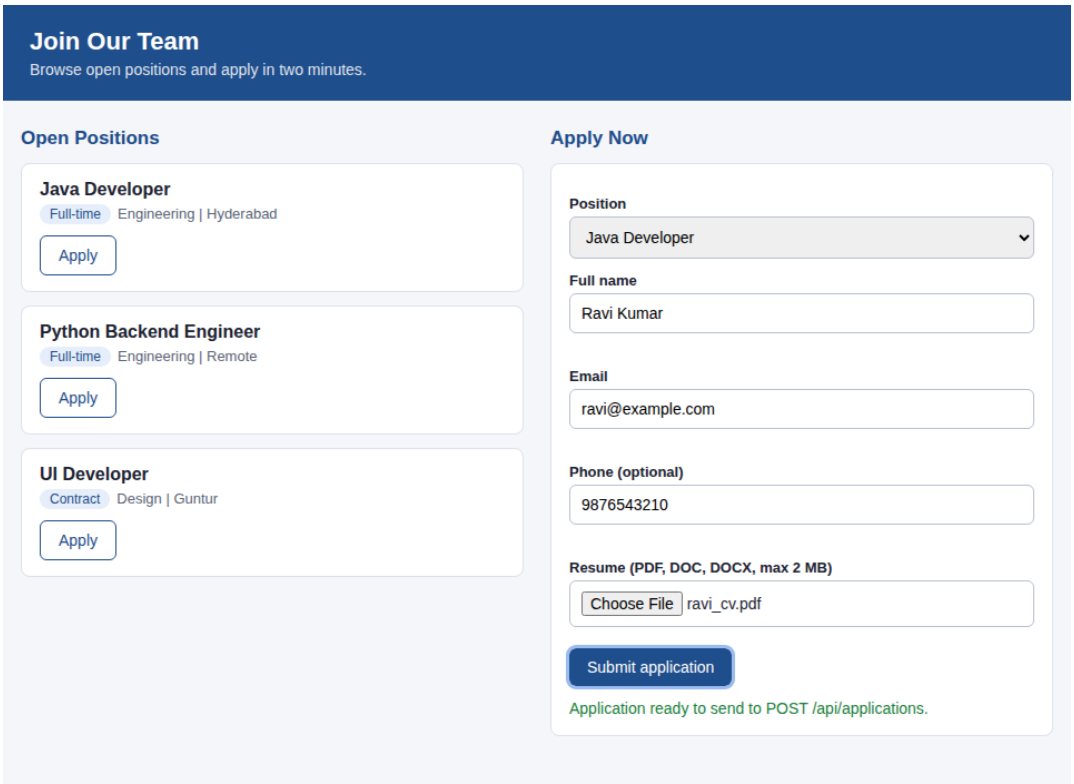


Figure B2. Valid input: no field errors and the ready-to-send message appears.

216. Test Cases Observed

The table lists the behaviours confirmed by the runs in this part. Results are copied from the program output shown on the preceding pages.

ID	Scenario	Expected	Observed	Layer
T1	Apply twice to the same job	Second attempt blocked	Blocked: already applied	Python, Java
T2	Move Applied to Hired	Blocked	Cannot move from Applied to Hired	Python, Java
T3	Move Hired to Screening	Blocked	Cannot move from Hired to Screening	Python, Java
T4	Full path Applied to Hired	Each step accepted, history saved	Four steps recorded	Python, Java
T5	Search candidates by skill 'Python'	Two matches	Meena Iyer, Sneha Reddy	Python (SQL)
T6	Invalid email 'ravi@'	Rejected	invalid	JavaScript
T7	Resume PNG, 5 MB	Rejected	Resume must be PDF, DOC or DOCX	JavaScript, HTML
T8	PDF larger than 2 MB	Rejected	Resume must be 2 MB or smaller	JavaScript
T9	Valid application data	No errors	{} (empty)	JavaScript, HTML

217. Limitations and Next Steps for Part B

- MySQL-specific statements were not run on a MySQL server in this environment; run both .sql files on MySQL 8 and compare the report results with the output shown here.
- The FastAPI app and JDBC class need a live database, so API responses on page 208 are examples, not captured logs.
- Authentication, password hashing, resume upload storage and role checks (Part A, sections 25 and 26) are not implemented in this code and must be added before real use.
- The careers page validates the form but does not yet send it to the API; the final step is a fetch POST to /api/applications with the resume sent as multipart form data.
- Add automated tests (pytest for Python, JUnit for Java) and a test MySQL database before deployment.

218. Conclusion of Part B

Part B turns the design in Part A into code that can be run and checked. The same pipeline rules produce identical results in Python and Java, the database enforces the most important constraints, and the front end gives users clear feedback before anything is sent to the server. With a MySQL server connected and the limitations above addressed, this code forms a working foundation for the Recruitment Applicant Tracking System.